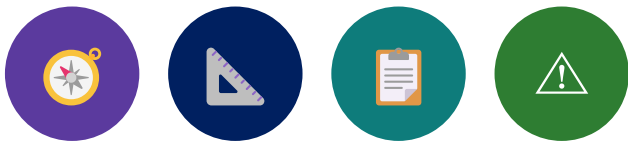


*"Freeze the plan before you freeze the code
-- architecture, ADRs, and a backlog
everyone can defend."*

MODULE 48

Capstone Planning and Architecture

Turn the Northstar CRM brief into an executable architecture and delivery plan: C4 context and container views, measurable NFRs, a prioritized vertical backlog, ADRs, and a scored risk register that Labs 49-52 build against.







MODULE 48 OVERVIEW

Produce C4 context and container views, measurable NFRs, a prioritized backlog, ADRs, and a scored risk register -- the plan Labs 49-52 build against.

Week 6 opens with the Northstar CRM capstone: a large enterprise needs a customer management platform for its service agents. Before any code is written, the team must freeze scope, architecture, NFRs, backlog, and risks -- reviewers will not fund disconnected demos.

DURATION & LAB



4 Hours Theory

Architecture views, requirements, NFR writing, backlog and ADR authoring, and risk planning.



2 Hours Lab

lab48-crm: context and container diagrams, NFRs, backlog, five or more ADRs, and a risk register.



Scenario

Northstar CRM for service agents -- Amina (CUS-1001) and Ravi (CUS-1002) as the fixed demo fixtures.

MODULE ARC



Architecture First

C4 context and container views before any backend or frontend code exists.



Vertical Backlog

CAP-12 and friends -- stories that cross API, UI, data, and events, never horizontal layers.



Scored Risk

Every risk gets a likelihood, an impact, a mitigation, and an owner -- never just 'hope'.

KEY TAKEAWAY Total time: 4 hours theory + 2 hours lab = 6 hours. No Lab 49-52 work counts as in scope unless it maps to a backlog item, an ADR, and a measurable NFR.

WHY ENTERPRISE PROJECTS NEED PLANNING

Leadership will not fund disconnected demos -- reviewers need traceability from business outcomes to architecture, stories, tests, and deployment.



Traceability or Rejection

Every Lab 49-52 change must trace to a backlog item, an ADR, or an explicit out-of-scope note.



Ambiguity Blocks Acceptance

Vague 'fast' or 'scalable' language and missing trust boundaries are acceptance-blockers at the Lab 52 defense.



Architecture Before Code

Context and container views, NFRs, and ADRs come before any Spring Boot or React work begins.



Graded Like Code

Lab 48 is scored on the same 100-point rubric rigor as any coding lab in this bootcamp.

KEY TAKEAWAY A peer must be able to reproduce the Week 6 plan from your docs alone -- pretty diagrams without trust boundaries or owners lose real marks.

Why Enterprise Projects Need Planning



MODULE LEARNING OBJECTIVES

By the end of this module, you will be able to plan an enterprise architecture a team can actually build against.

1. Clarify business scope, users, journeys, exclusions, and success measures.
2. Produce C4 context diagrams with protocols and trust boundaries.
3. Design container views placing React, Spring Boot, PostgreSQL, Kafka, identity, and observability.
4. Define domain ownership and versioned HTTP and event contracts.
5. Write measurable NFRs with method, environment, and thresholds.
6. Slice a vertical backlog with acceptance criteria, including story CAP-12.
7. Record architecture decisions as ADRs with alternatives and consequences.
8. Plan ownership, critical path, and a scored risk register with mitigations.

KEY TAKEAWAY These eight objectives are drawn directly from the graded lab -- you will produce all six core artifacts: context, containers, NFRs, backlog, ADRs, and risk register.



Module Learning Objectives



CAPSTONE PROJECT OVERVIEW

Northstar CRM is a customer management platform for service agents -- one coherent product, not five disconnected demos.

Field	Value
Codename	Northstar CRM
Java Package	com.northstar.crm
Repo Path	~/java-bootcamp/examples/customer-management-platform/
Primary Users	Service agent, manager, platform operator

DEMO FIXTURES

**Amina Khan -- CUS-1001**

ACTIVE -- the primary demo customer for the interaction timeline.

**Ravi Singh -- CUS-1002**

PROSPECT to ACTIVE -- the onboarding and status-change journey.

**CUS-9999**

Not-found and negative-path testing -- never a real customer.

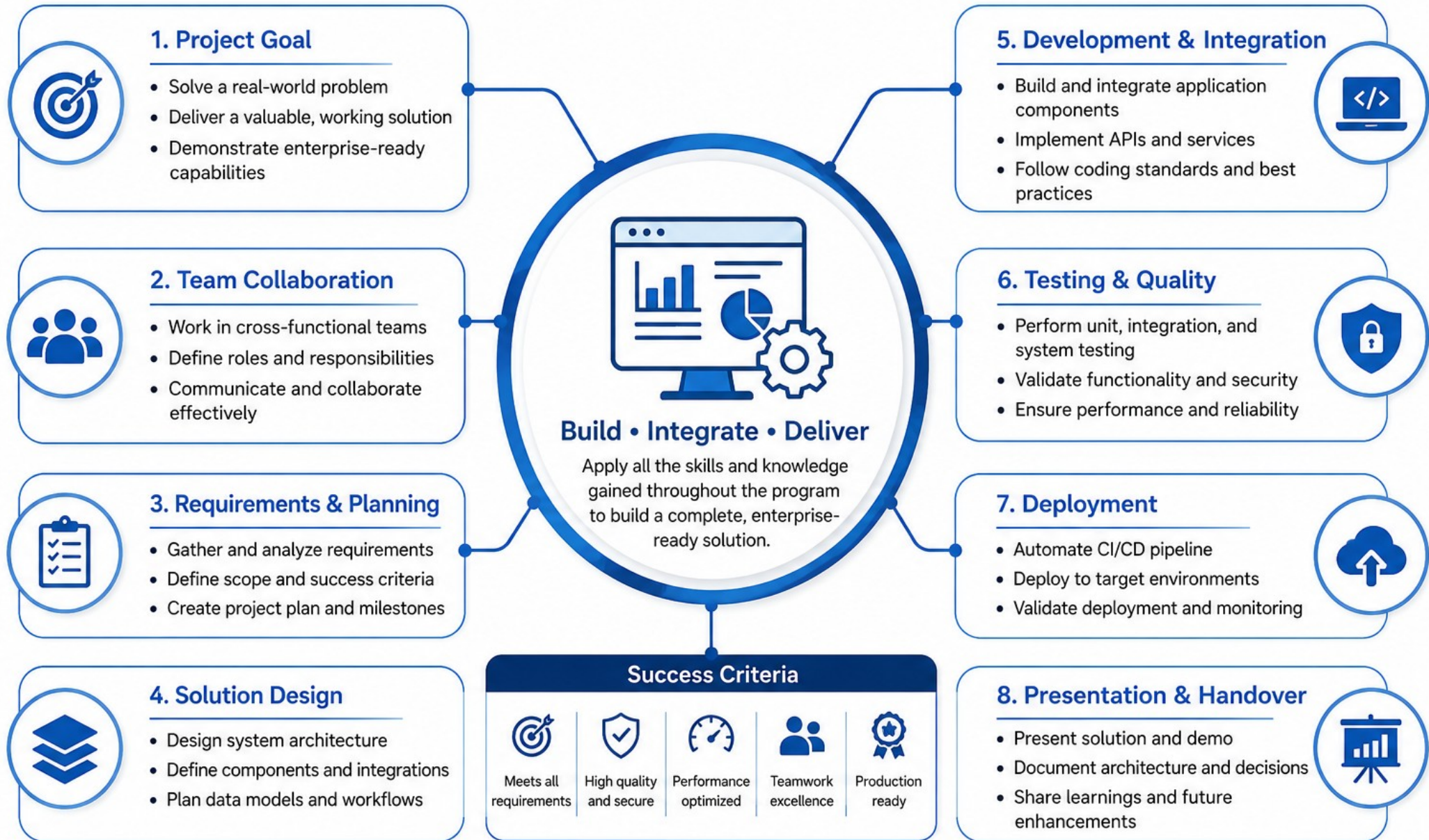
**lab-request-001**

The correlation ID threaded through every API call, event, and log.

KEY TAKEAWAY Fixture IDs CUS-1001, CUS-1002, CUS-9999, and lab-request-001 must appear consistently in every planning doc and every later lab.



Capstone Project Overview



UNDERSTANDING BUSINESS REQUIREMENTS

Without named users, journeys, exclusions, and success measures, 'done' is contested in the Lab 52 defense panel.

WHAT TO CLARIFY

Primary users: service agent, manager, platform operator.

Journeys: search Amina/Ravi, view profile/timeline, record interaction.

In-scope capabilities vs. explicit exclusions (billing, real PII import).

Success measures tied to the Lab 52 demo and NFR thresholds.

OPEN QUESTIONS DISCIPLINE

- Every open question gets an owner and a due date.
- Vague 'build a CRM' gets rewritten with users and measurable success.
- Real customer names get replaced with synthetic fixtures immediately.
- A peer must be able to paraphrase the outcome statement in one pass.

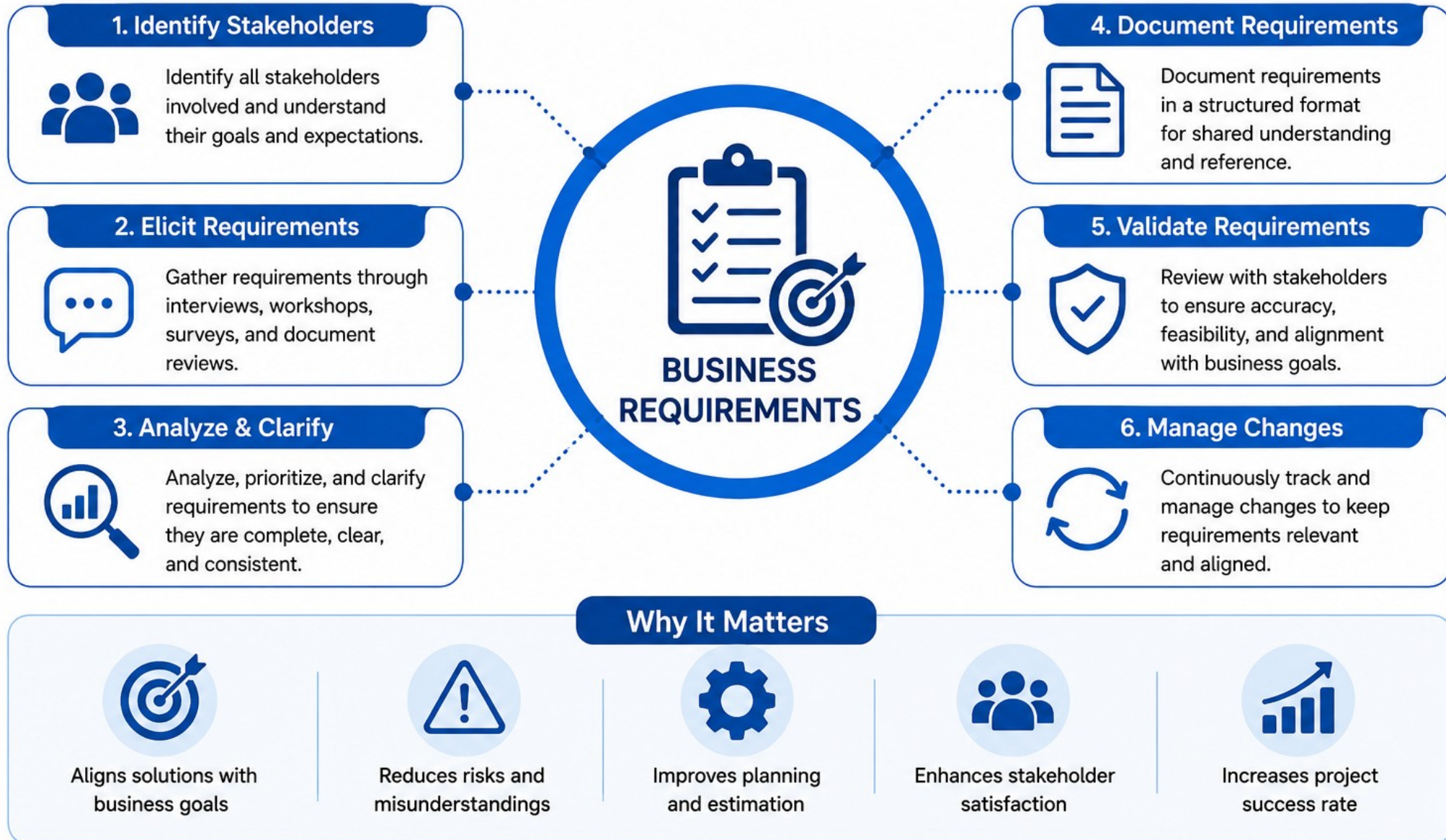
context.md outcome statement (excerpt)

```
Users: service agent, manager, operator.  
Journeys: search -> profile -> record interaction for CUS-1001.  
Excluded: billing, real PII import, mobile app.
```

KEY TAKEAWAY A one-page outcome statement a peer can paraphrase, with exclusions explicit and every open question owned.

Understanding Business Requirements

The foundation of every successful solution.



FUNCTIONAL / NON-FUNCTIONAL REQUIREMENTS

Functional requirements describe what the system does; non-functional requirements describe how well it must do it -- both need concrete acceptance criteria.

FUNCTIONAL REQUIREMENTS

Search and open a customer profile (Amina, Ravi).

Record a customer interaction (story CAP-12).

Change customer status (PROSPECT to ACTIVE).

View a customer's interaction timeline.

NON-FUNCTIONAL REQUIREMENTS (NFRs)

- Latency: p95 create-interaction API $\leq$ 500 ms in the lab.
- Availability: API readiness within 3 minutes after deploy.
- Security: unauthenticated /api/** returns 401; wrong role returns 403.
- Accessibility: keyboard-complete interaction form with associated labels.

Banned vs. required language

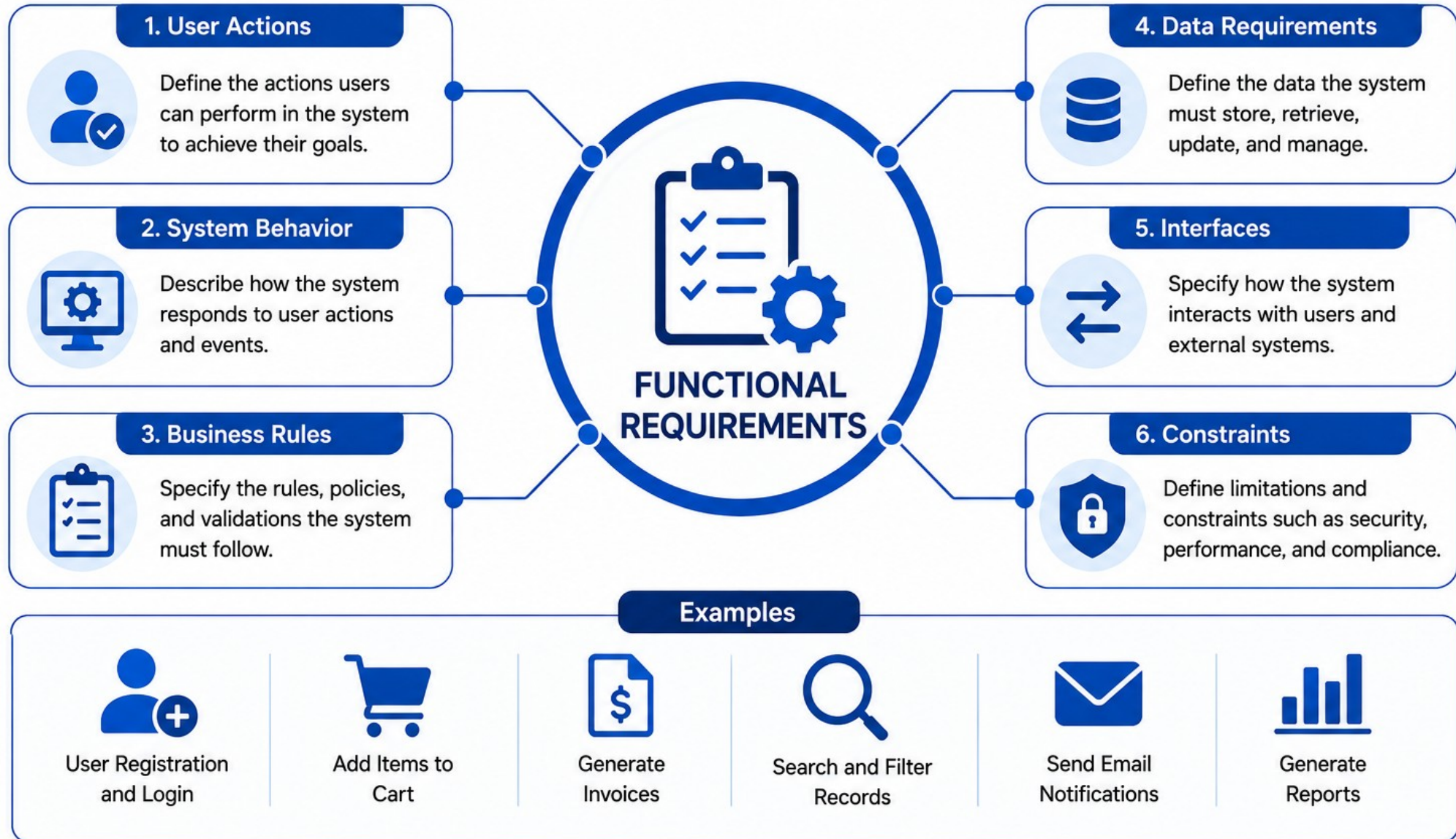
Banned: "the system should be fast and secure."

Required: "p95 latency $\leq$ 500 ms, measured via Micrometer, in the lab environment."

KEY TAKEAWAY 'Fast,' 'secure,' and 'scalable' without a threshold, method, and environment cannot be tested or defended.

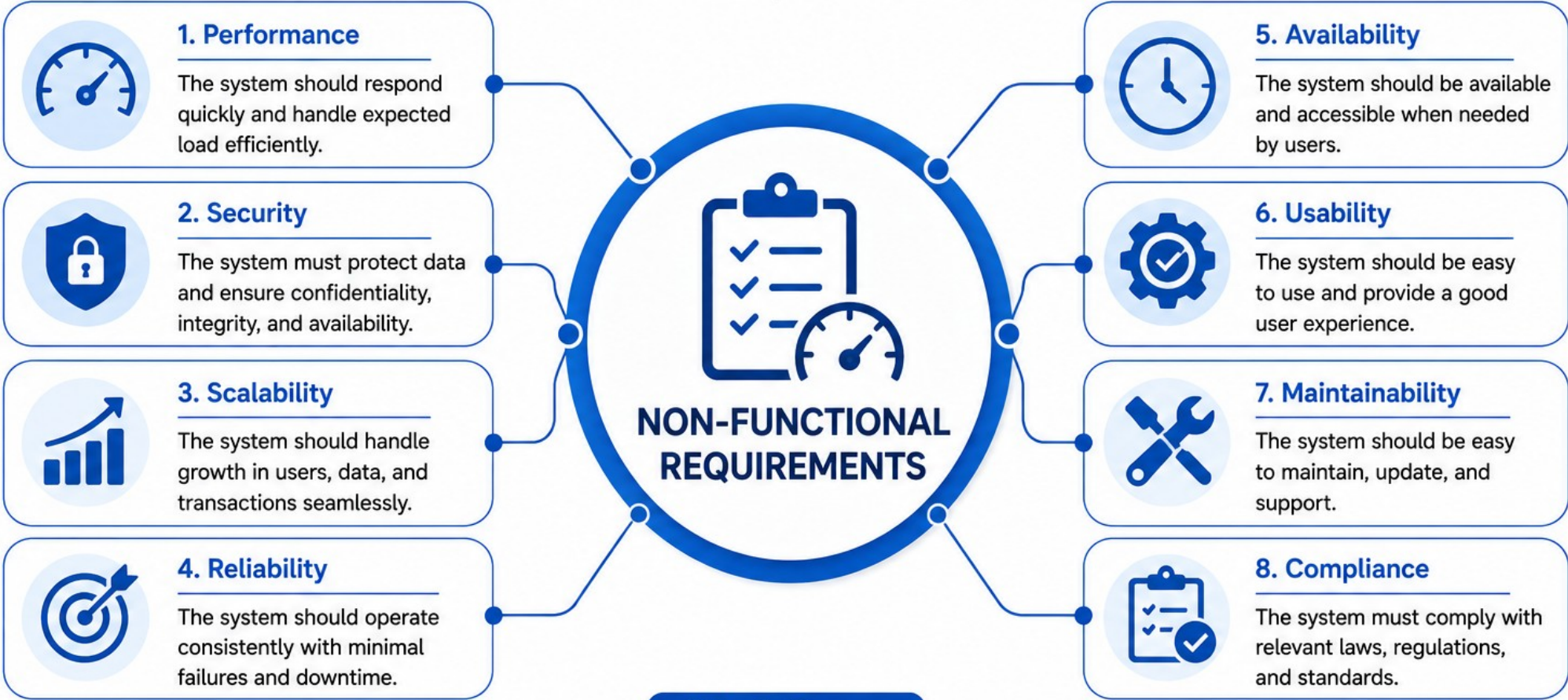
Functional Requirements

- Define **what** the system should do. •

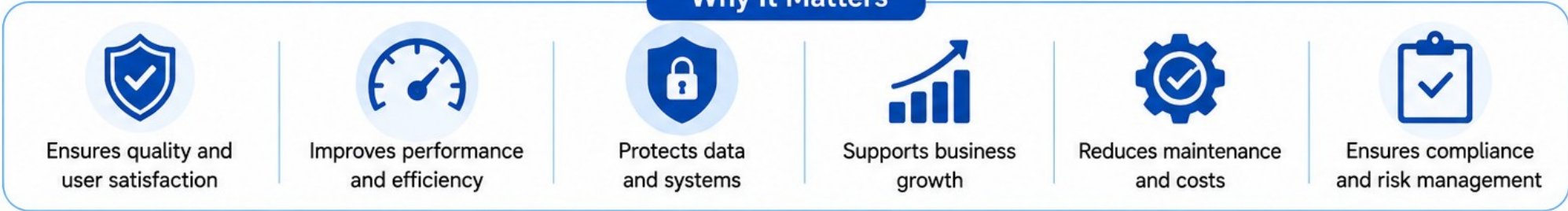


Non-Functional Requirements

Define **how** the system should perform.



Why It Matters



STAKEHOLDER EXPECTATIONS

Service agents, managers, and platform operators need different things from the same Northstar CRM release.

STAKEHOLDER NEEDS

Service agents: fast search, reliable interaction recording.

Managers: visibility into backlog priority and demo readiness.

Platform operators: safe deploy, rollback, and observability.

Instructors (Lab 52 panel): reproducible evidence, not claims.

MANAGING EXPECTATIONS

- State exclusions explicitly -- do not let scope creep in silently.
- Tie every promise to a backlog item or an ADR, never a verbal aside.
- Escalate blockers early, using the same rules learned in Module 47.
- Never present an unverified claim as fact to any stakeholder.

Traceability chain

```
Business outcome -> architecture choice -> backlog story -> test  
-> deployment -> operations.
```

KEY TAKEAWAY Every stakeholder promise must be traceable to a backlog item or an ADR -- disconnected demos do not satisfy any of them.

Stakeholder Expectations



ENTERPRISE ARCHITECTURE FUNDAMENTALS

C4 model views separate concerns so each audience sees the right level of detail.

Level	What It Shows
Context	People and external systems -- protocols and trust boundaries only.
Container	Deployable units -- React, Spring Boot, PostgreSQL, Kafka, the IdP.
Component	Modules within a container -- controllers, services, repositories.
Code	Classes and interfaces -- explicitly out of scope for Lab 48 diagrams.

WHY LEVELS MATTER



Context = No Internals

Keep class names and Kafka topic internals out of the context view.



Container = Deployment Units

Show synchronous REST+JWT and asynchronous event flows, both labeled.



Component = Later

Component and code views can wait until Lab 49 implementation begins.



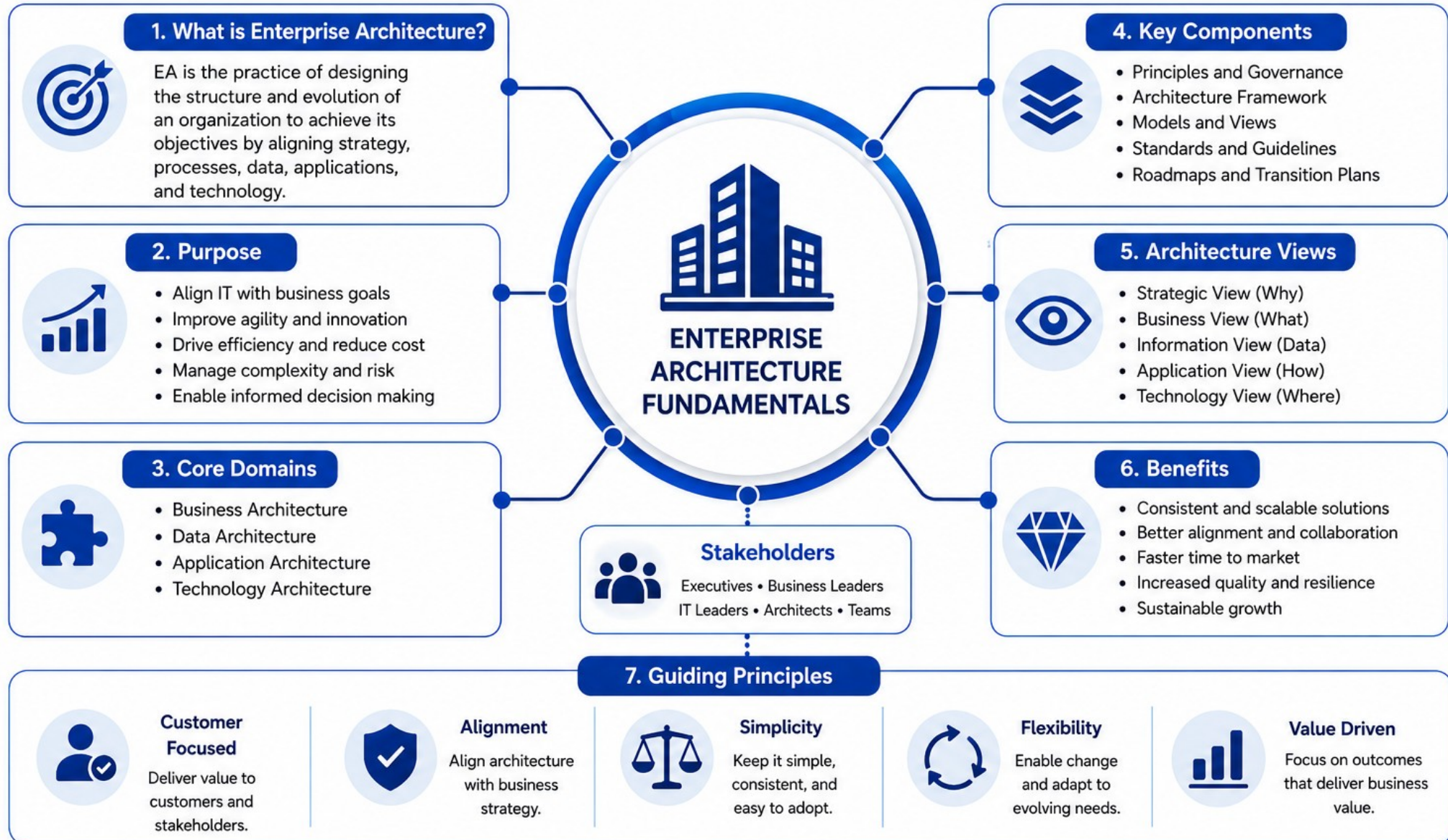
Trust Boundaries Everywhere

Mark where JWT auth, the database, and Kafka cross a trust zone.

KEY TAKEAWAY Context diagrams without trust boundaries hide IdP and data-exfiltration risks reviewers will probe in Lab 52.

Enterprise Architecture Fundamentals

A blueprint for aligning business strategy with technology.



IDENTIFYING SYSTEM COMPONENTS

Container placement forces the sync-versus-async decisions that ADRs and NFRs must later quantify.

CORE CONTAINERS

React CRM UI -- the agent-facing journey.

Spring Boot API -- REST + JWT resource server.

PostgreSQL -- system of record via Spring Data JPA.

Kafka + notification/worker consumer -- async side effects.

SUPPORTING SYSTEMS

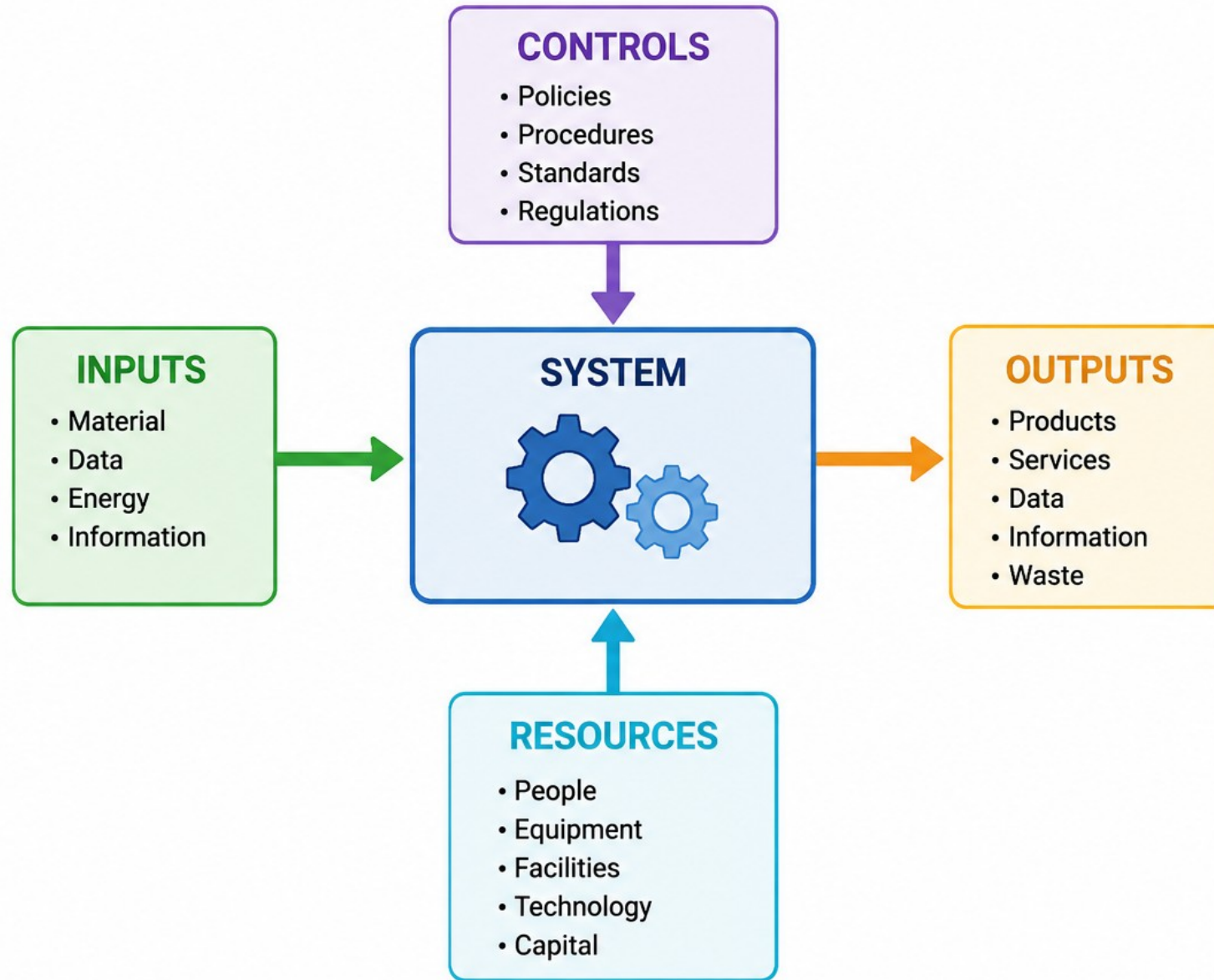
- Identity Provider -- OIDC/JWT issuance and validation.
- Logs / metrics / traces -- the observability container.
- Admin/deployment boundary -- who may apply k3s manifests.
- An orphan Kafka topic needs a documented consumer or a risk note.

container.md sync/async labels

```
UI --REST+JWT--> API --JPA/JDBC--> PostgreSQL
API --Customer events--> Kafka --> Notification Consumer
```

KEY TAKEAWAY Missing PostgreSQL or an orphan Kafka topic with no consumer are named Lab 48 failure modes -- add the container or document the risk.

Identifying System Components



TRY IT YOURSELF -- EXERCISE 1: SKETCH CONTEXT DIAGRAM

Reinforces: identifying users, external systems, and trust boundaries for Northstar CRM before any container-level detail is added.

STEPS (notes/lab48-context-sketch.md)

Step	What To Do
1 -- Actors	List service agents, admins, and external IdP/email/Kafka dependencies.
2 -- Check the Reference	Confirm docs/architecture/context.md and container.md are the target files.
3 -- Trust Boundaries	Mark where JWT auth, the database, and Kafka cross a trust zone.
4 -- Fixtures	Note CUS-1001/CUS-1002 as demo data, not external systems.

Context sketch skeleton

```
Actors: Service Agent, Manager, Platform Operator, Identity Provider.  
Trust boundary: browser vs. API vs. IdP vs. Kafka consumers.
```

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 1 before continuing: exercises/exercise-01-context-sketch.md (workspace: examples/module-48-exercises).

FRONTEND / BACKEND / MESSAGING / DATABASE DESIGN

Four containers, four responsibilities -- each with its own contract to the others.



Frontend

React CRM UI -- agent search, profile, timeline, and interaction form over REST + JWT.



Backend

Spring Boot API -- layered controller/service/repository, Problem Details errors.



Messaging

Kafka topic `crm.customer.interactions.v1` -- `CustomerInteractionRecordedV1` events.

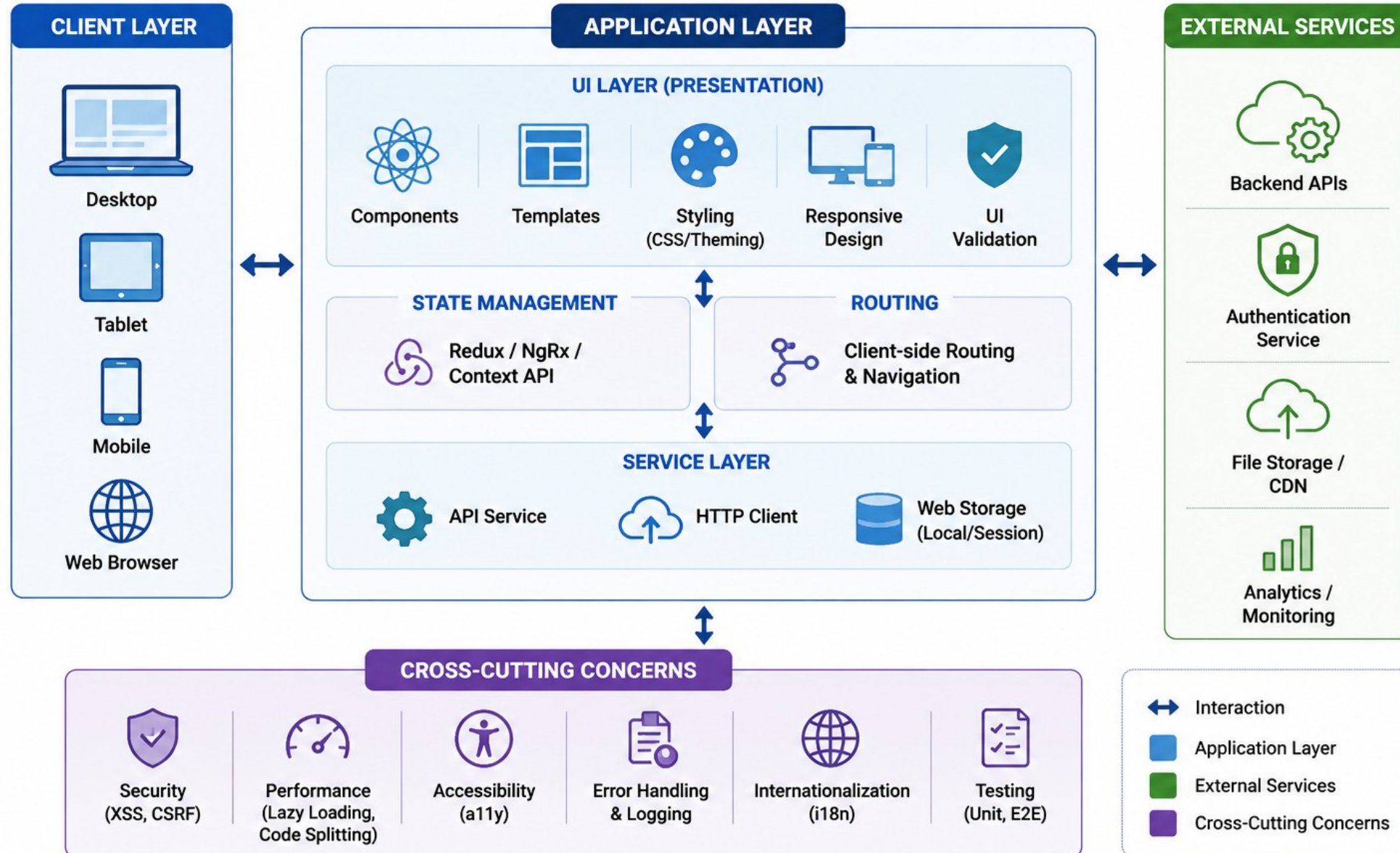


Database

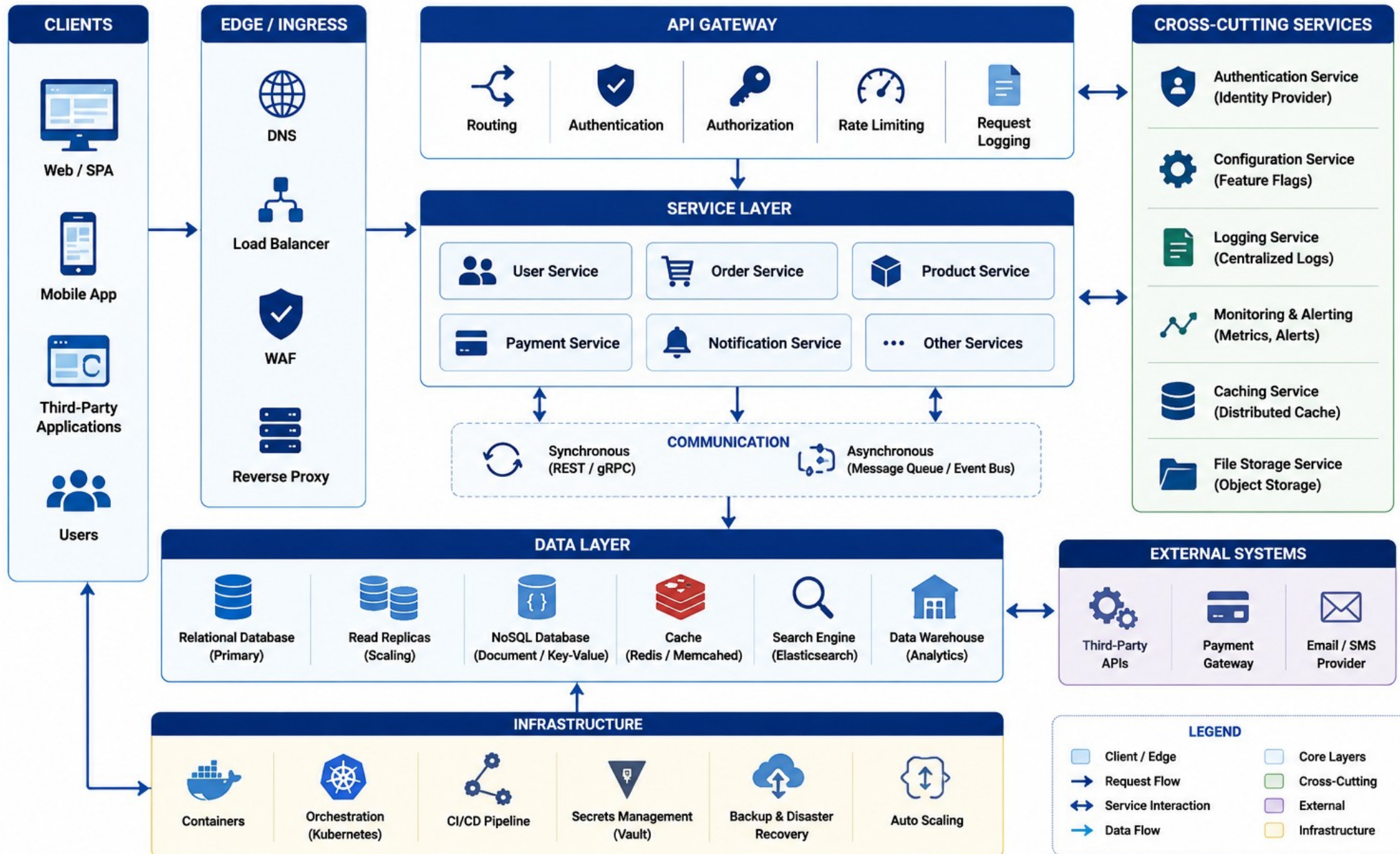
PostgreSQL system of record via Spring Data JPA -- migrations, not 'Hibernate will figure it out'.

KEY TAKEAWAY Each layer gets its own ADR-worthy decisions -- and Lab 49 implements exactly the backend and messaging halves of this picture.

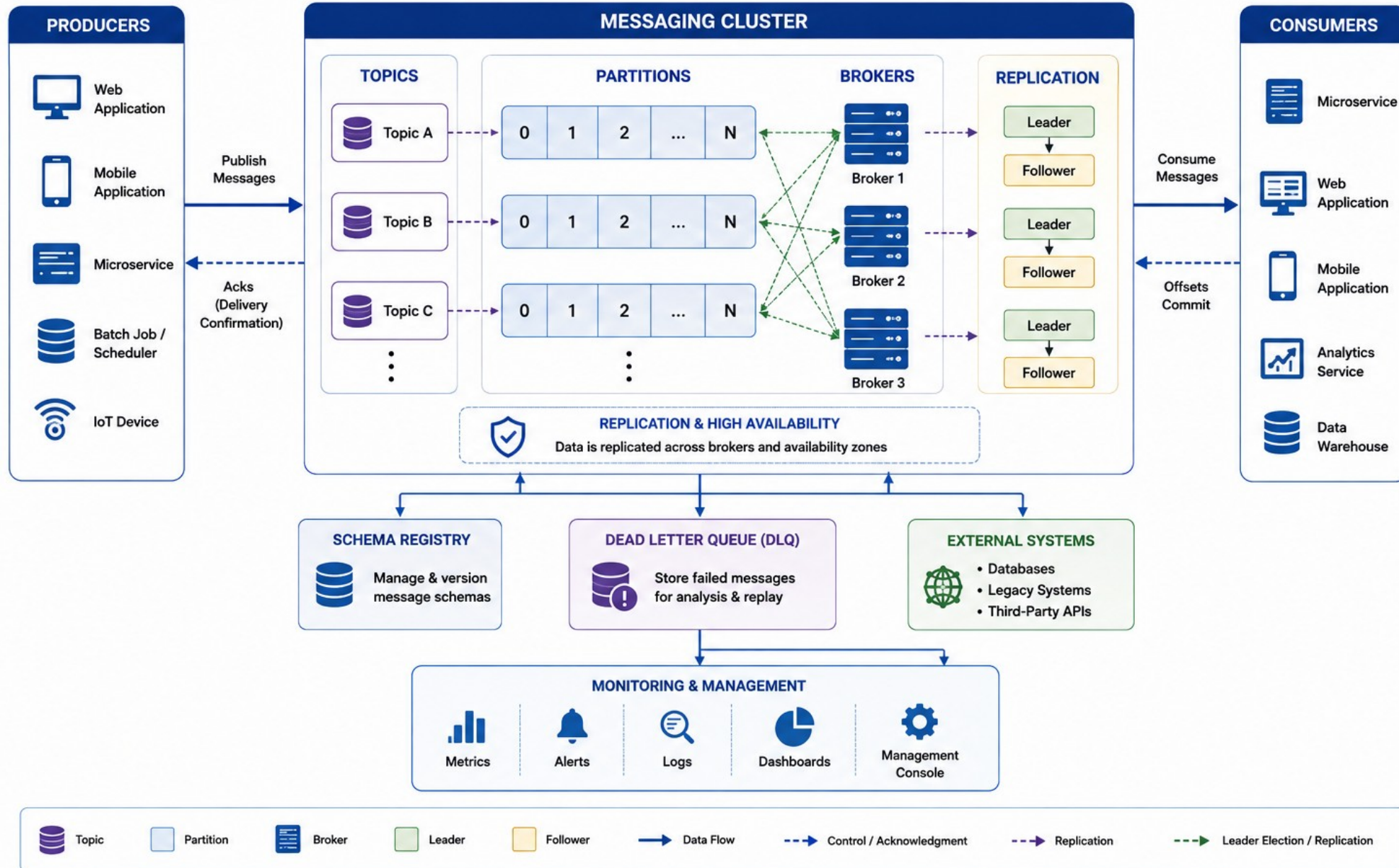
Frontend Architecture Design



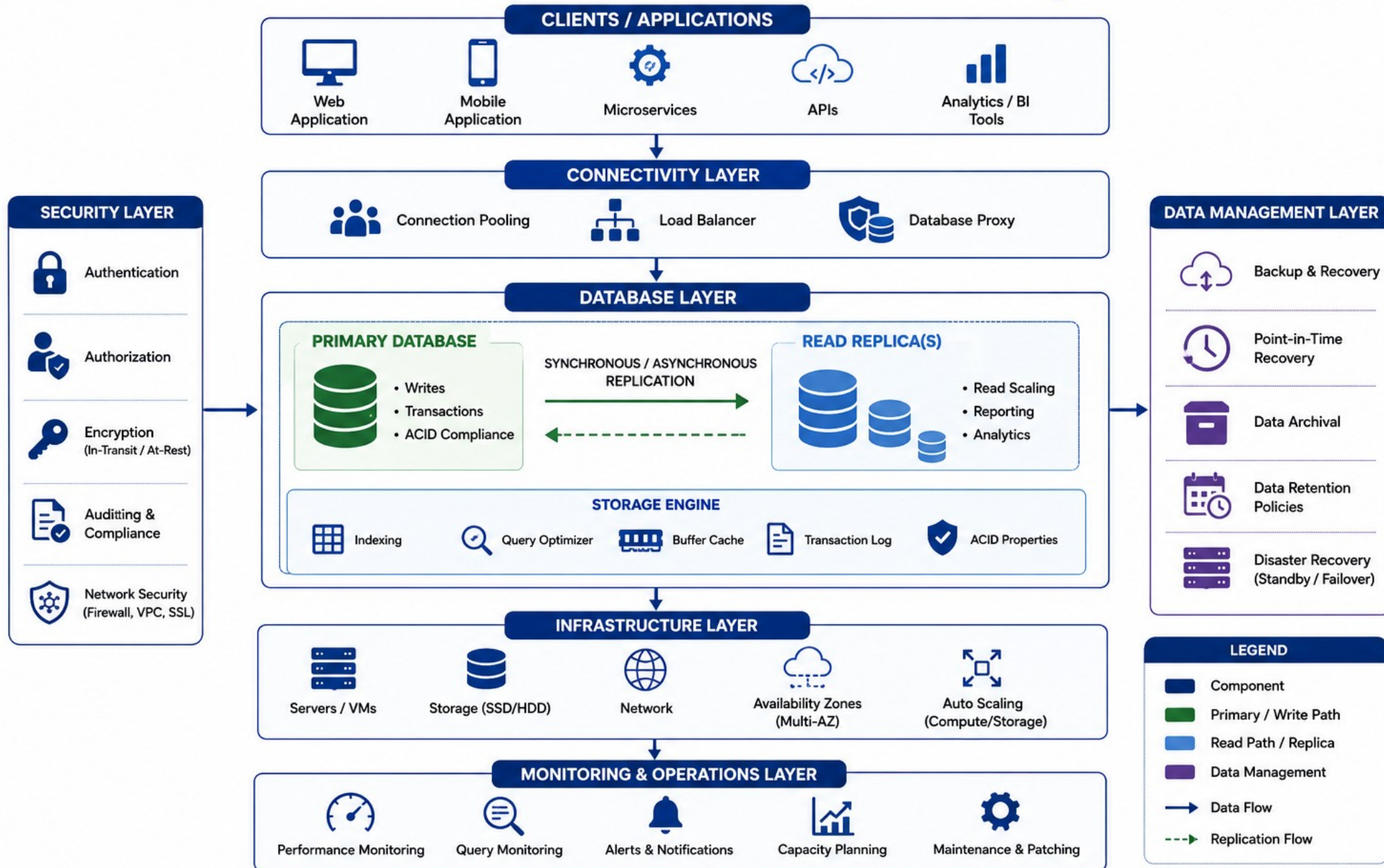
Backend Architecture Design



Messaging Architecture Design



Database Architecture Design



SELECTING THE TECHNOLOGY STACK

The Northstar CRM stack is fixed for Week 6 -- naming it precisely now avoids drift across Labs 49-51.

APPLICATION STACK

React (Vite, Node 22) for the agent-facing UI.

Spring Boot (JWT/RBAC) for the API layer.

PostgreSQL + Spring Data JPA for persistence.

Kafka for customer interaction events.

DELIVERY STACK

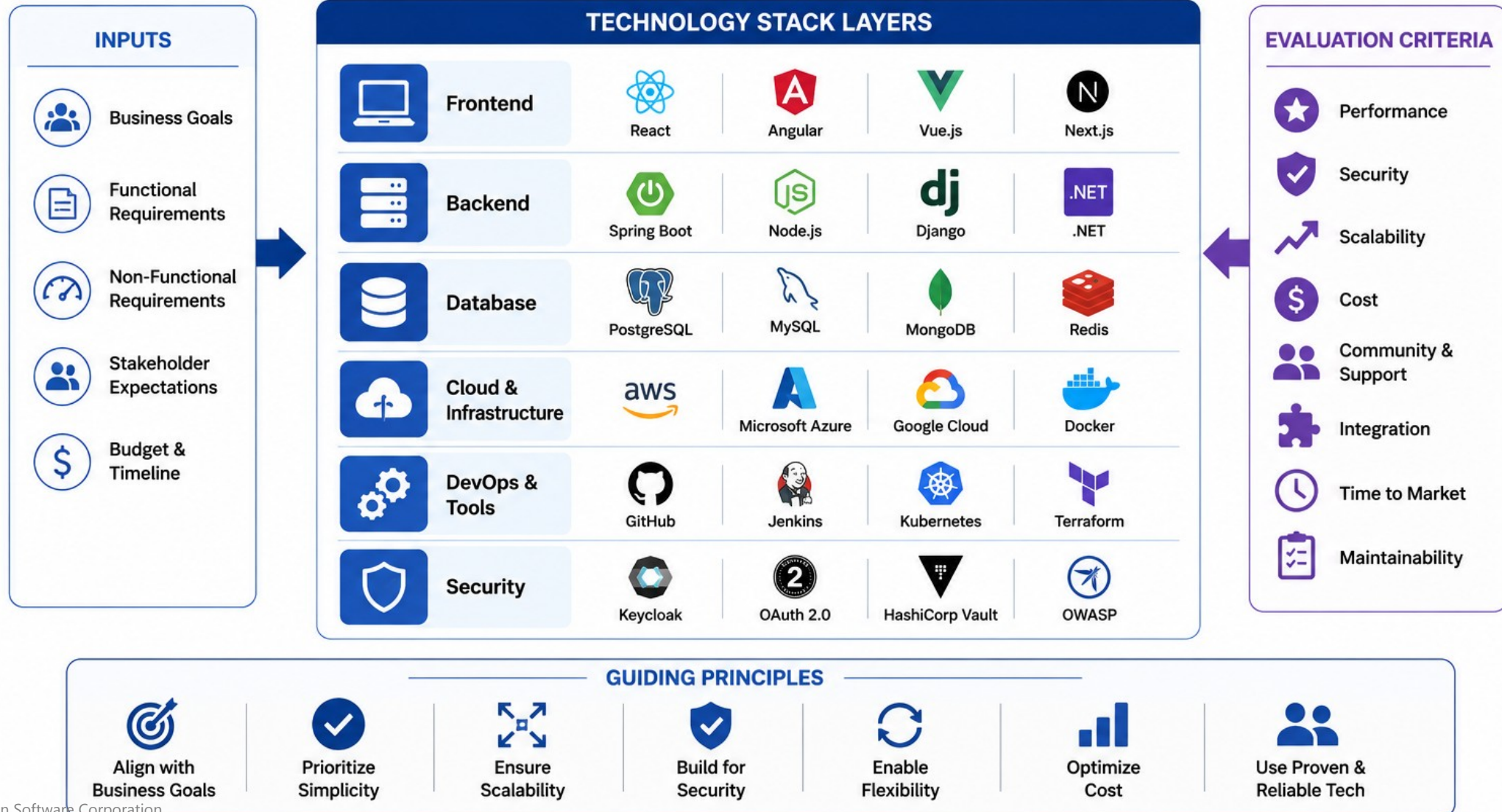
- Docker for immutable images.
- k3s (training cluster) for deployment; OpenShift as an enterprise comparison.
- GitHub Actions for CI/CD; Bitbucket Pipelines as an enterprise comparison.
- JUnit / Mockito for automated verification.

Stack-to-lab map

```
Lab 49: Spring Boot + Kafka.   Lab 50: React + PostgreSQL/JPA.  
Lab 51: Docker + GitHub Actions + k3s.   Lab 52: defense + retrospective.
```

KEY TAKEAWAY Naming the stack once here prevents Lab 49-51 from re-arguing technology choices mid-week.

Selecting the Technology Stack



DEFINING SYSTEM BOUNDARIES

A clear boundary tells Labs 49-52 what is this week's problem and what is explicitly deferred.

IN SCOPE

Customer search, profile, and interaction timeline.

Recording an interaction (story CAP-12) end to end.

Status change: PROSPECT to ACTIVE.

A secured, deployed release with rollback evidence.

OUT OF SCOPE

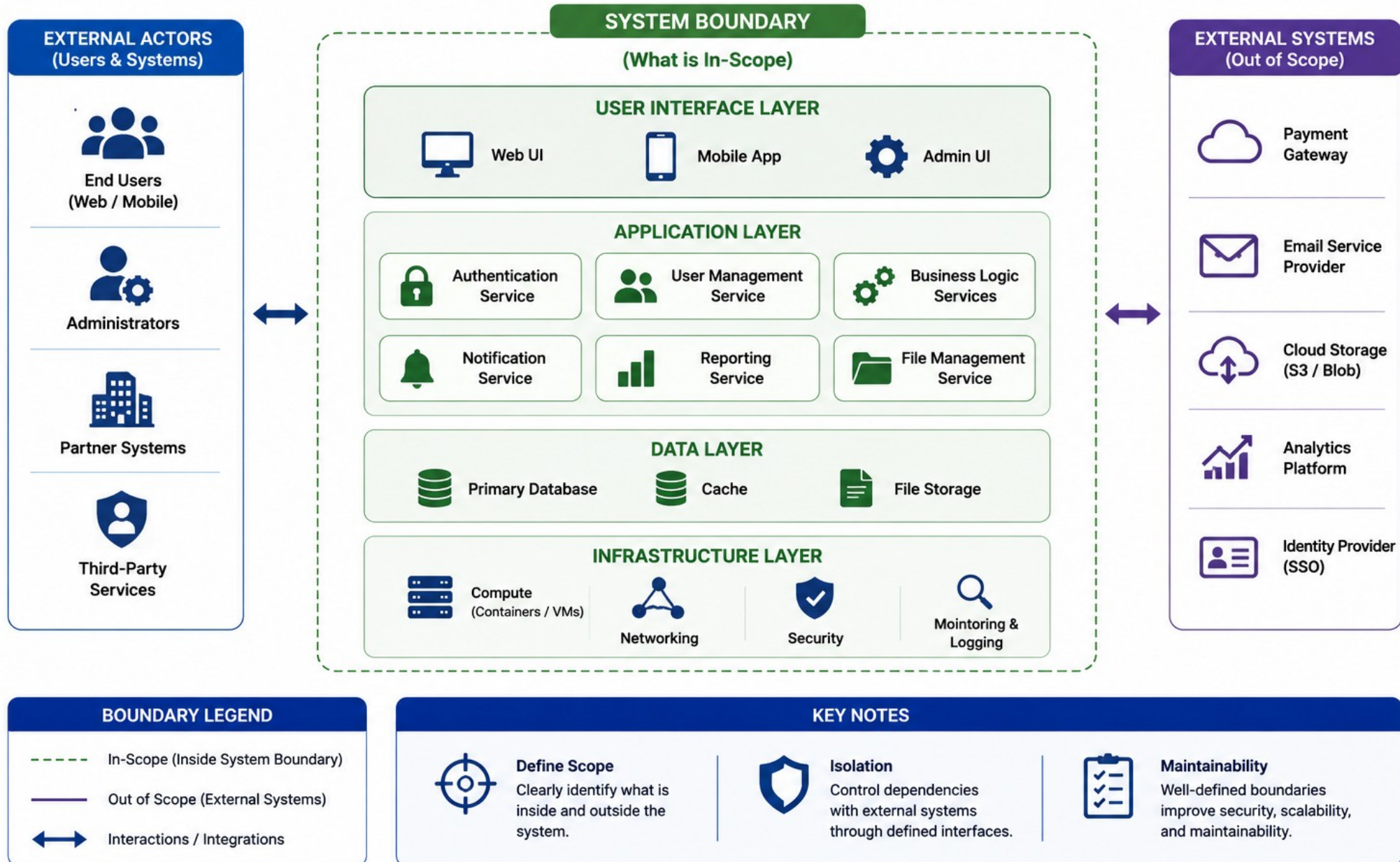
- Billing and payments.
- Real customer PII import from production systems.
- A native mobile app.
- Multi-tenant white-labeling.

Explicit deferral note

```
"Billing is explicitly out of scope for Week 6 -- deferred, not silently dropped."
```

KEY TAKEAWAY If a story is deferred, mark it Explicitly Deferred with an owner and date in the risk register -- never silently drop it.

Defining System Boundaries



IDENTIFYING INTEGRATION POINTS

Every integration point is a place contract drift or a trust-boundary failure can hide.

INTEGRATION POINTS

UI <-> API over REST + JWT.

API <-> PostgreSQL over JPA/JDBC.

API <-> Kafka for customer events.

Pipeline <-> registry <-> cluster for deploy.

CONTRACT DISCIPLINE

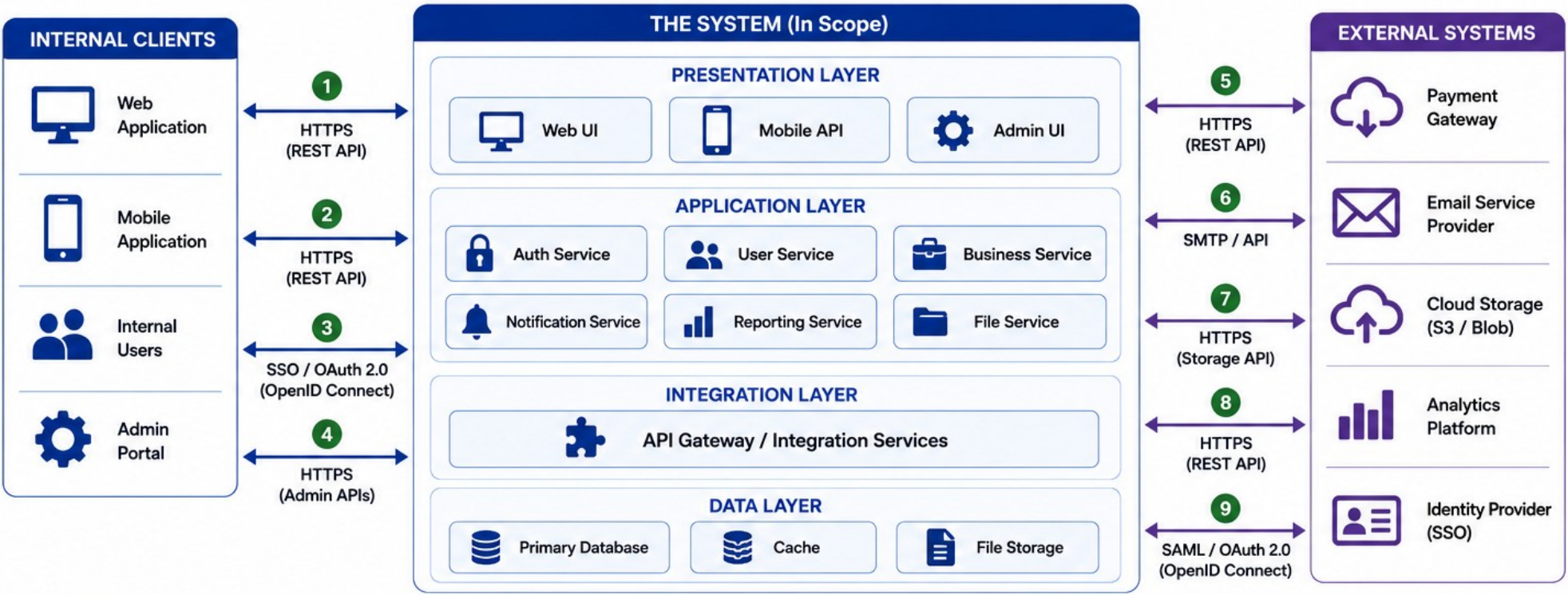
- Draft POST /api/customers/{id}/interactions with Problem Details errors.
- Draft CustomerInteractionRecordedV1 fields before Lab 49 codes them.
- Additive fields are fine; breaking changes require a version bump.
- Every request carries header X-Correlation-ID: lab-request-001.

Event sketch (fields only)

```
eventId, eventType, eventVersion, occurredAt,  
correlationId, actor, customerId, interactionId, channel
```

KEY TAKEAWAY Unversioned endpoints and events create Lab 50 contract breakage and Lab 52 panel failure -- version from day one.

Identifying Integration Points



INTEGRATION POINTS SUMMARY					
ID	Integration Point	Type	Protocol / Technology	Direction	Purpose
1	Web Application ↔ System	API	HTTPS (REST API)	↔	Access system features and data
2	Mobile Application ↔ System	API	HTTPS (REST API)	↔	Access system features and data
3	Internal Users ↔ System (SSO)	Security	SSO / OAuth 2.0 (OIDC)	↔	Authenticate and authorize users
4	Admin Portal ↔ System	API	HTTPS (Admin APIs)	↔	System administration
5	System ↔ Payment Gateway	API	HTTPS (REST API)	↔	Process payments and refunds
6	System → Email Provider	Service	SMTP / API	→	Send emails and notifications
7	System ↔ Cloud Storage	API	HTTPS (Storage API)	↔	Store and retrieve files
8	System → Analytics Platform	API	HTTPS (REST API)	→	Send events and metrics
9	System ↔ Identity Provider	Security	SAML / OAuth 2.0 (OIDC)	↔	SSO authentication and user info

LEGEND	
	Internal Components (In Scope)
	External Systems (Out of Scope)
	Integration Point ID
	Bi-directional Integration
	Outbound Integration
	External Integration
	Integration Layer (Orchestration)

SECURITY ARCHITECTURE PLANNING

Security decisions made now save Lab 51 from a redesign under deadline pressure.

Concern	Lab 48 Decision
Authentication	OIDC/JWT resource-server pattern, decided via ADR.
Authorization	Role-based -- AGENT and MANAGER roles at minimum.
Secrets	Never in ADRs, screenshots, or committed docs.
Untrusted Input	Browser agents, JWT claims, and Kafka payloads are all untrusted.

SECURITY PLANNING HABITS



JWT/OIDC ADR Now

Decide the auth pattern in Lab 48 -- Lab 51 hardens, it does not redesign.



Role-Based, Not Person-Based

Plan AGENT/MANAGER roles, never per-person special cases.



Fictional Emails Only

amina.khan@example.test, never a real email, in any planning doc.

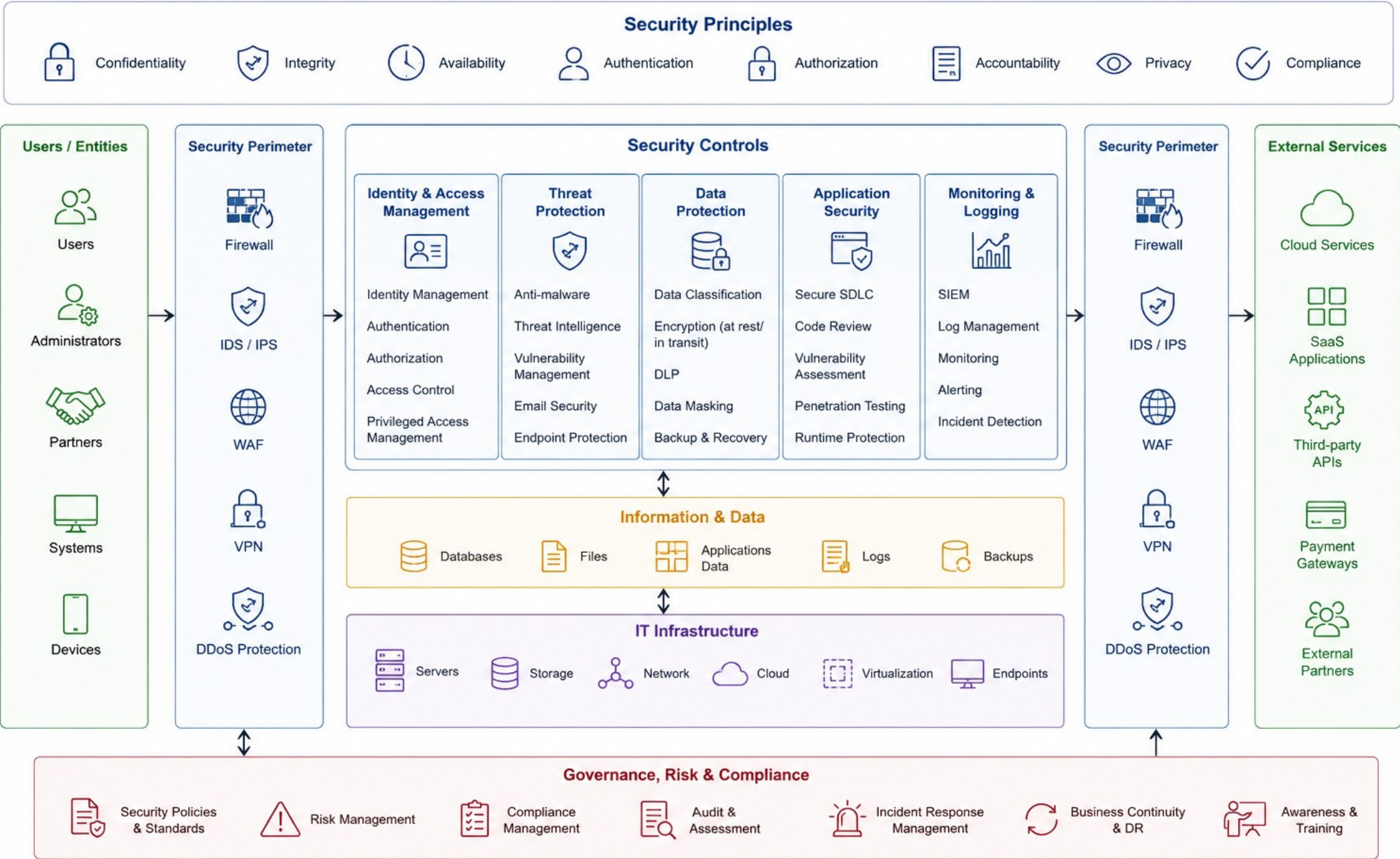


No Secrets In Docs

Never paste real IdP secrets, kubeconfigs, or production URLs.

KEY TAKEAWAY Which inputs are untrusted, where authn/authz is enforced, and which values are sensitive -- answer all three before Lab 49 starts coding.

Security Architecture Planning



SCALABILITY / RELIABILITY CONSIDERATIONS

Scalability and reliability are NFRs too -- they need thresholds and a measurement method, not adjectives.

SCALABILITY

Partition Kafka events by customer ID for ordered processing.
Avoid N+1 queries when loading interaction timelines.
Index PostgreSQL for timeline-by-customer-and-time lookups.
Document which profile targets PostgreSQL vs. H2 in CI.

RELIABILITY

- Rollback the previous digest within 10 minutes (rehearsed in Lab 51).
- Consumers must be idempotent -- duplicate events are normal.
- Bounded retries plus a dead letter topic for poison messages.
- API readiness probe within 3 minutes after deploy.

NFR line (reliability)

```
"Rollback previous digest <= 10 min, method: rehearse in Lab 51, environment: k3s training namespace."
```

KEY TAKEAWAY Every scalability or reliability claim needs a number, a measurement method, and an environment -- or it is not an NFR yet.

Scalability Considerations

SCALABILITY PRINCIPLES



Think Ahead
Anticipate growth and plan capacity



Horizontal Scale
Scale out, not up



Decouple
Reduce dependencies between components



Automate
Use automation for provisioning & scaling



Monitor
Measure, alert and optimize continuously



Cost Efficient
Optimize resources and control costs



Resilient
Design for failure and self-healing

SCALE DIMENSIONS



Users
Support more concurrent users



Throughput
Handle more requests per second



Data Volume
Manage growing amounts of data

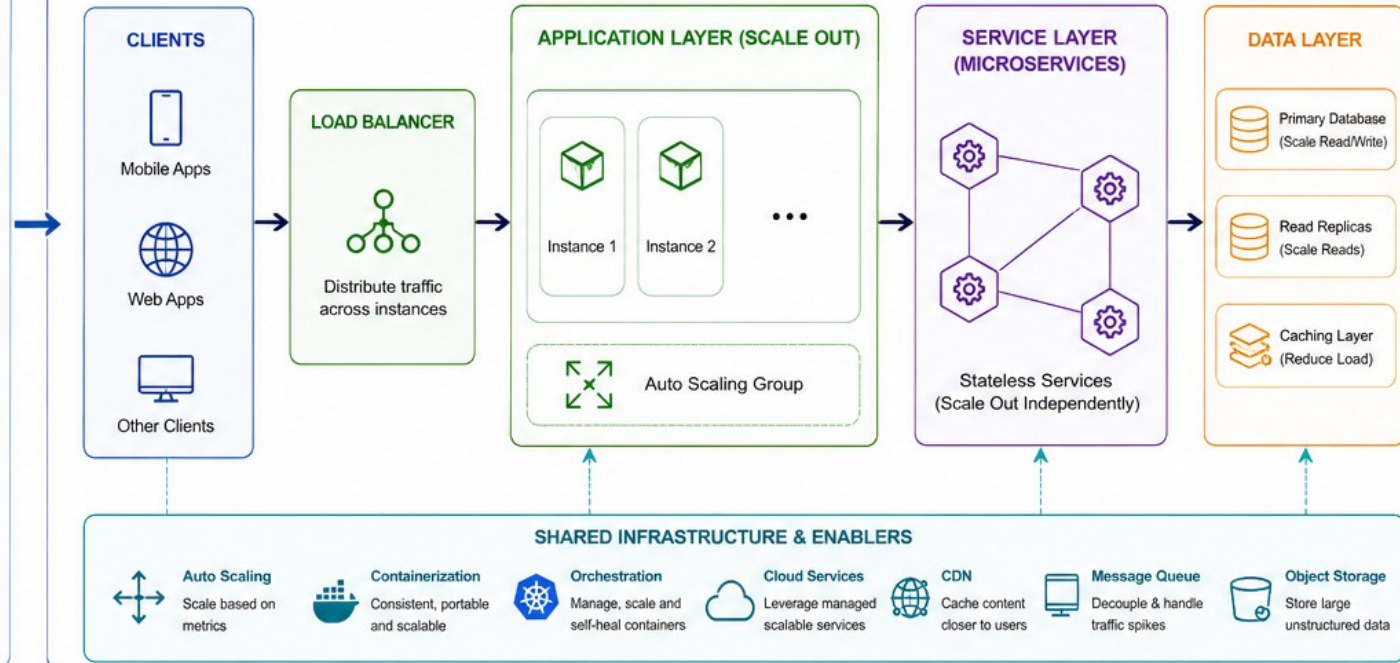


Geographic
Serve users globally with low latency



Features
Scale by adding new capabilities

SCALABLE ARCHITECTURE PATTERN



SCALABILITY STRATEGIES



Horizontal Scaling
Add more instances



Database Scaling
Sharding, Replication, Partitioning



Caching
In-memory caching, CDN



Asynchronous Processing
Queues, Events, Background jobs



Stateless Design
Keep services stateless for easy scale out

KEY CONSIDERATIONS



Capacity Planning
Forecast growth and plan resources



Bottleneck Identification
Find and eliminate scaling bottlenecks



Performance Testing
Load test to validate at scale



Monitoring & Observability
Monitor metrics, logs and traces



Resilience & High Availability
Multi-AZ/Region, failover and self-healing



Cost Management
Right-size resources and optimize usage

OUTCOMES



High Performance
Fast response at any scale



High Availability
Minimal downtime and disruptions



Cost Efficient
Better resource utilization



Future Ready
Easily adapt to future growth

Reliability Considerations

RELIABILITY PRINCIPLES



Availability
Maximize uptime and meet SLA



Dependability
Consistent, reliable behavior



Fault Tolerance
Continue operating despite failures



Resilience
Recover quickly from disruptions



Recoverability
Restore service and data



Consistency
Maintain correct state



Observability
Visibility to detect and resolve issues

RELIABILITY INPUTS



Business Requirements
SLA, RTO, RPO, uptime targets



Workload Characteristics
Traffic patterns, peak loads, growth



Dependency Mapping
Internal & external dependencies



Risk Assessment
Identify potential failure scenarios



Historical Data
Incidents, trends, lessons learned

RELIABLE SYSTEM ARCHITECTURE

CLIENTS

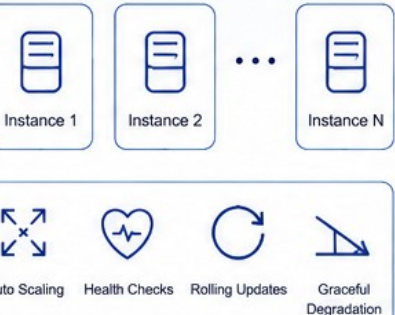


LOAD BALANCER

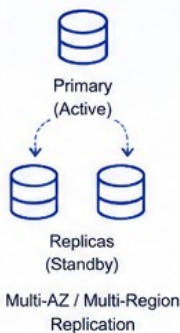


Distribute traffic across healthy instances
Health Checks

APPLICATION LAYER



DATA LAYER



EXTERNAL SERVICES



FAULT TOLERANCE MECHANISMS



Redundancy
Eliminate single points of failure



Replication
Synchronous / Asynchronous



Failover
Automatic failover



Health Checks
Detect failures early



Timeouts
Prevent hanging requests



Retries
Handle transient failures



Circuit Breaker
Prevent cascading failures

RELIABILITY OUTCOMES



High Availability
Meet or exceed uptime targets



Resilient Service
Sustain operations during failures



Fast Recovery
Minimize downtime and data loss



Consistent Service
Reliable performance and correct results



Customer Trust
Dependable and predictable service

RELIABILITY PRACTICES



Capacity Planning
Plan for growth and peak loads



Performance Testing
Load, stress, spike and endurance tests



Chaos Engineering
Test system behavior under failure



Monitoring & Alerting
Real-time alerts and thresholds



Logging & Metrics
Collect and analyze key metrics



Incident Management
Detect, respond and resolve quickly



Post-Incident Review
Learn and improve continuously



Backup & DR Testing
Regular backups and DR drills

KEY RELIABILITY METRICS



Availability
(%)



MTBF
(Mean Time Between Failures)



MTTR
(Mean Time To Recover)



Error Rate
(%)



Throughput
(Requests/sec)



SLA Compliance
(%)



Change Failure Rate
(%)

STANDARDS & FRAMEWORKS



SRE Principles • ITIL • ISO 22301
NIST • Cloud Well-Architected Framework

TRY IT YOURSELF -- EXERCISE 2: DRAFT MEASURABLE NFRS

Reinforces: replacing vague quality words with measurable NFR placeholders across security, traceability, recoverability, performance, and operability.

STEPS (notes/lab48-nfr-placeholders.md)

Step	What To Do
1 -- Categories	Security, traceability, recoverability, performance, operability -- one NFR each.
2 -- Check the Reference	Each NFR needs metric + target + how you will prove it in Labs 49-52.
3 -- Example Rewrite	Rewrite 'system should be fast' into a measurable statement with a placeholder number.
4 -- Scope	Placeholders only -- the full NFR doc is completed in Lab 48.

Rewrite pattern

```
Vague: "the system should be fast."  
Measurable: "p95 latency <= ____ ms, measured via ____, in ____."
```

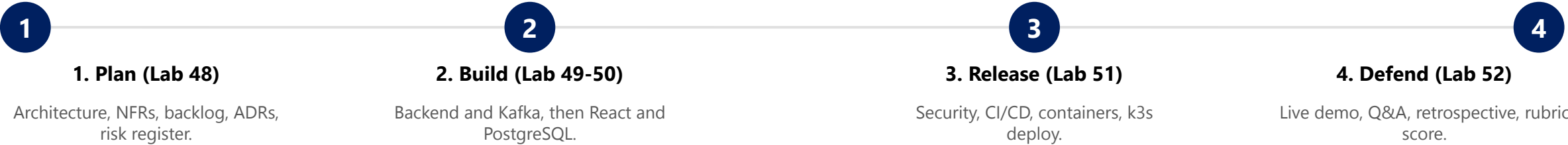
KEY TAKEAWAY TRY IT NOW -- Complete Exercise 2 before continuing: exercises/exercise-02-nfr-placeholders.md (workspace: examples/module-48-exercises).

AGILE PROJECT PLANNING

The capstone runs on the same Agile rhythm as every prior module, scaled up to a five-lab delivery week.

Role	Responsibility	Communicates With
Product Owner	Owns backlog priority for CAP-12 and friends.	Stakeholders and the team.
Scrum Master	Facilitates process, removes blockers.	Team and leadership.
Development Team	Delivers Labs 49-51's vertical slices.	Product Owner and QA.
Release Commander	Owns the Lab 51 deploy and rollback decision.	Ops, engineers, stakeholders.

WEEK 6 SPRINT CYCLE



KEY TAKEAWAY Kanban visualizes flow with a work-in-progress limit; this capstone timeboxes flow into five labs -- both exist to keep delivery predictable.

Agile Project Planning

AGILE PRINCIPLES



Customer Satisfaction



Welcome Change



Deliver Value Frequently



Work Together



Motivated Individuals



Face to Face Communication



Working Software



Sustainable Pace



Continuous Improvement

1. VISION & INITIATION



Define Vision

- Product Vision
- Goals & Objectives
- Success Criteria



Identify Stakeholders

- Customers
- Users
- Team
- Sponsors



High-Level Scope

- Key Features
- Constraints
- Assumptions



Project Charter

- Purpose
- High-Level Plan
- Budget & Timeline (Rough)

2. RELEASE & ITERATION PLANNING



Product Backlog

Prioritized list of all features, user stories, bugs, and improvements

Feature / Epic 1

Feature / Epic 2

User Story 1

User Story 2

...

Technical Debt

Bug Fixes



Release Planning

Define release goals, scope, and timeline

Release Goal

Select & Prioritize

- Business Value
- Dependencies
- Capacity

Release Roadmap

R1 R2 R3



Iteration (Sprint) Planning

Plan work for the next iteration (1-4 weeks)

Sprint Goal

Select Stories

- From Top of Backlog
- Based on Priority
- Fit to Capacity



Sprint Backlog

Tasks, Sub-tasks
Estimates
Acceptance Criteria



Scrum Master



Product Owner

Agile Team

Developers

QA / Tester

3. ITERATIVE DELIVERY (SPRINT)



Review

1 - 4 Weeks Sprint

Design

Build

Test

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

Design

Build

Test

Review

USER STORIES AND BACKLOG CREATION

Horizontal 'layers first' backlogs strand Week 6 without a demoable vertical slice -- every story must cross API, UI, data, and events lightly.

VERTICAL STORY RULE

Order by value, risk, dependency, and learning -- not by layer.

Every story needs numbered acceptance criteria, not just a task title.

Enabling tech stories must cite which outcome they unlock.

Amina/Ravi fixtures appear in acceptance criteria, not random data.

CAP-12 WORKED EXAMPLE

- As a service agent, I want to record an interaction for CUS-1001.
- AC1: valid input returns 201 + a resource id; correlation preserved.
- AC2: the timeline shows the interaction within two seconds of refresh.
- AC4: invalid notes return field-level errors and are not persisted.

CAP-12 -- Record a customer interaction

```
As a service agent, I want to record an interaction for CUS-1001
(Amina Khan) so the next agent understands customer history.
```

KEY TAKEAWAY CAP-12 is the backbone story Lab 49 implements end to end -- get its acceptance criteria right here, or Lab 49 improvises.

User Stories and Backlog Creation

1. WHAT IS A USER STORY?

A user story is a short, simple description of a feature from the perspective of the user.

User Story Template

As a <type of user>,
I want <an action>,
so that <a benefit>.

Example



As a registered user,
I want to reset my password
so that I can regain access
to my account.

Characteristics



Independent
Can stand on its own



Negotiable
Open to discussion and change



Valuable
Provides value to the user/business



Estimable
Can be estimated in size



Small
Small enough to complete in a sprint



Testable
Can be verified

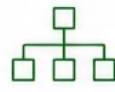
2. FROM IDEA TO USER STORIES

1. Gather Requirements



- Interviews
- Workshops
- Surveys
- Existing Systems
- Feedback

2. Identify Epics



Group related requirements into epics (large features).

3. Write User Stories



Break epics into user stories using the user story template.

4. Define Acceptance Criteria



- Clear
- Testable
- Measurable
- Agreed by all

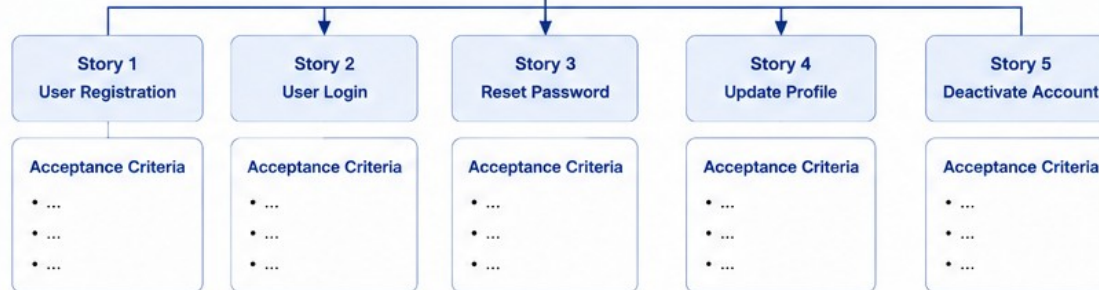
5. Estimate (Sizing)



Estimate effort/size using planning poker, T-shirt sizing, or story points.

EXAMPLE: EPIC BREAKDOWN

Epic: User Account Management



4. BACKLOG REFINEMENT (ONGOING)

1. Review & Clarify



Ensure understanding of the story, intent, and value.

2. Add Details



Elaborate acceptance criteria, notes, and edge cases.

3. Re-estimate



Refine size as more information becomes available.

4. Re-prioritize



Adjust priority based on value, risk, and dependencies.

5. Ready for Sprint



Stories at the top are ready for sprint planning.

3. BACKLOG CREATION

Product Backlog



A prioritized list of user stories and other items needed to deliver value.

Backlog Items

US-001

As a registered user, I want to reset my password so that I can regain access to my account.

5

US-002

As a new user, I want to register an account so that I can use the system.

8

US-003

As a user, I want to update my profile so that my information is accurate.

3

...
Higher Priority
Lower Priority

5. BENEFITS



Clear understanding of user needs



Better estimation and planning



Prioritized delivery of value



Improved collaboration



Higher customer satisfaction

TRY IT YOURSELF -- EXERCISE 3: SKETCH VERTICAL STORIES

Reinforces: writing three vertical stories that cross API/UI/data/events lightly, with acceptance criteria and Must/Should priority.

STEPS (notes/lab48-backlog-slice.md)

Step	What To Do
1 -- Stories	Create customer, record interaction, search + timeline -- using Amina/Ravi fixtures.
2 -- Check the Reference	Stories need acceptance criteria -- not tasks like 'make controller'.
3 -- Priority	Order Must/Should and note dependencies on Labs 49-51.
4 -- Scope	Three stories only -- the full backlog is completed in Lab 48.

Story skeleton

```
As a <role>, I want <capability> so <outcome>.  
Acceptance criteria: 1) ... 2) ... 3) ...
```

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 3 before continuing: exercises/exercise-03-backlog-slice.md (workspace: examples/module-48-exercises).

TASK BREAKDOWN STRUCTURE

A vertical story still needs a concrete task list -- broken down without losing its vertical shape.

BREAKING DOWN CAP-12

DTOs + Bean Validation for the interaction request.
Flyway migration for the customer_interaction table.
Transactional InteractionService with correlation propagation.
REST endpoint + Kafka publisher + consumer + tests.

BREAKDOWN RULES

- Tasks stay inside one story -- do not silently expand scope.
- Each task should be completable in under a day.
- Tests are a task, not an afterthought.
- Documentation (backend-demo.md) is a task, not optional.

Task list excerpt (CAP-12)

```
1) DTOs    2) Migration    3) Service    4) Endpoint  
5) Publisher  6) Consumer    7) Tests    8) backend-demo.md
```

KEY TAKEAWAY A good breakdown turns 'implement CAP-12' into eight completable tasks without losing the vertical, demoable shape.

Task Breakdown Structure



What is a TBS?

A Task Breakdown Structure (TBS) is a hierarchical decomposition of the total project work into smaller, manageable tasks.



Improves
Planning



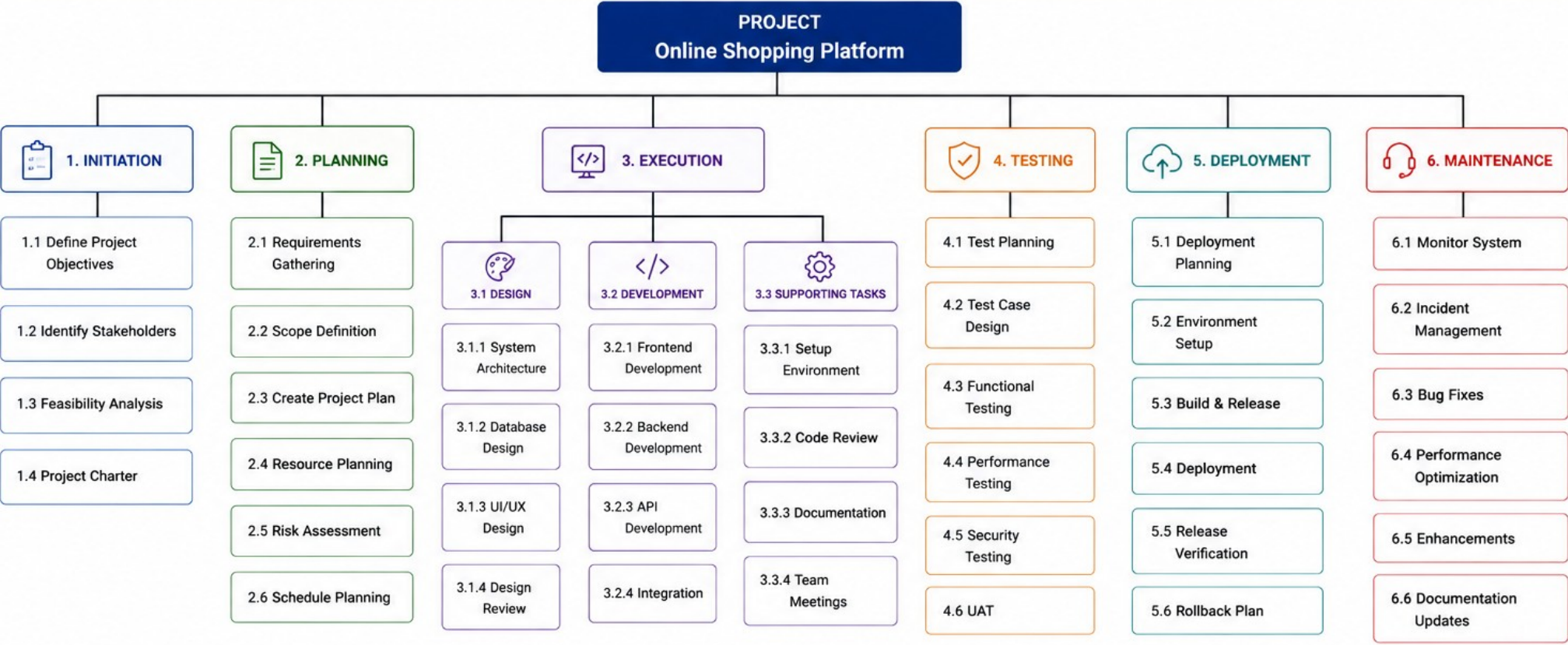
Enhances
Estimation



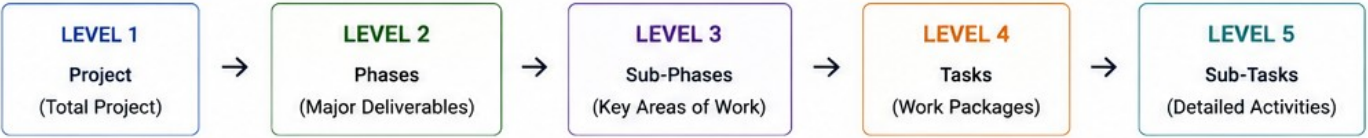
Clarifies
Ownership



Increases
Visibility



TBS LEVELS EXAMPLE



KEY POINTS

- ✓ Break work down to manageable tasks
- ✓ Each task should have a clear owner
- ✓ Tasks should be measurable and estimable
- ✓ Decompose until the right level of detail is achieved

TEAM ROLES AND RESPONSIBILITIES

Named owners, not a diffuse 'the team,' keep Labs 49-52 from renegotiating scope mid-week.



Lab 49 Owner

CAP-12 API + Kafka for CUS-1001, with correlation lab-request-001.



Lab 50 Owner

React search/profile/timeline + PostgreSQL durability proof.



Lab 51 Owner

JWT deny-by-default, pipeline gates, immutable image, rollback.



Lab 52 Owner

Demo script, evidence index, Q&A cards, retro, self-assessment.

KEY TAKEAWAY Write this ownership checklist into docs/team-plan.md so Week 6 does not thrash over who owns what.

Team Roles and Responsibilities



Collaborate
Work together as one team



Deliver Value
Focus on customer value



Own Results
Take ownership and accountability



Communicate
Open, transparent communication



Adapt
Embrace change and improve



Respect
Value every contribution

ROLE	PRIMARY RESPONSIBILITIES	KEY ACCOUNTABILITIES	KEY SKILLS / COMPETENCIES	WORKS CLOSELY WITH
 Product Owner	- Define product vision and strategy - Manage and prioritize the product backlog - Represent stakeholders and customer needs - Accept or reject work items	- Value delivery - Backlog ROI and prioritization - Clear requirements and acceptance - Stakeholder satisfaction	- Business analysis - Prioritization & decision making - Communication & negotiation - Domain knowledge	 Stakeholders Scrum Master Development Team
 Scrum Master	- Facilitate Agile ceremonies - Remove impediments for the team - Coach the team on Agile principles - Promote continuous improvement	- Team effectiveness - Process adherence - Impediment resolution - Continuous improvement	- Agile practices - Facilitation - Coaching & mentoring - Problem solving	 Product Owner Development Team Organization
 Development Team	- Design, build, test, and deliver increments - Collaborate to meet sprint goals - Participate in planning and estimation - Maintain code quality and documentation	- Commitments - Quality of deliverables - Velocity and predictability - Technical excellence	- Technical expertise - Collaboration - Problem solving - Adaptability	 Product Owner Scrum Master QA / Testers
 QA / Tester	- Create test strategy and test plans - Design and execute test cases - Identify defects and verify fixes - Ensure quality standards	- Defect detection - Test coverage - Quality of releases - Timely feedback	- Testing techniques - Automation tools - Attention to detail - Analytical thinking	 Development Team Product Owner DevOps
 DevOps Engineer	- Build and maintain CI/CD pipelines - Manage infrastructure and deployments - Monitor systems and ensure availability - Automate processes and tools	- Deployment frequency - System reliability - Build & release quality - Incident response time	- CI/CD & Automation - Cloud & Infrastructure - Scripting / IaC - Monitoring & Security	 Development Team QA / Testers Operations
 UI/UX Designer	- Understand user needs and behaviors - Create wireframes, prototypes, and designs - Ensure usability and accessibility - Collaborate on user-centered solutions	- Usability - Design quality - User satisfaction - Consistent UX	- UI/UX Design - Prototyping tools - User research - Communication	 Product Owner Development Team QA / Testers
 Business Analyst	- Elicit and document requirements - Analyze processes and data - Create user stories and use cases - Validate solution meets business needs	- Requirement accuracy - Traceability - Business alignment - Stakeholder communication	- Requirements analysis - Modeling & documentation - Communication - Domain knowledge	 Product Owner Development Team Stakeholders

SPRINT PLANNING FOR CAPSTONE

Five labs, five days -- sprint planning here is really a critical-path plan across the whole capstone week.

CRITICAL PATH

Lab 48 architecture and backlog gate everything downstream.

Lab 49 backend/Kafka must land before Lab 50's UI can integrate.

Lab 51 security/deploy needs a working Lab 49-50 slice.

Lab 52 defense needs evidence from every prior lab.

PLANNING DISCIPLINE

- Time-box diagramming and ADR writing -- they often burn 90 minutes.
- Require one complete vertical story early, not five partial ones.
- Track milestones with named owners and due dates.
- Re-plan explicitly if a story is deferred -- never silently drop it.

Milestone line

```
"CAP-12 API+Kafka green by end of Lab 49 -- owner: <name>, due: Day 2."
```

KEY TAKEAWAY The critical path runs Plan -> Backend -> Frontend -> Release -> Defense -- a slip anywhere upstream delays the whole chain.

Sprint Planning for Capstone



PURPOSE

Define sprint goal, select capstone backlog items, estimate, and create a commitment plan.

OUTCOMES



Clear Sprint Goal



Selected & Prioritized
Backlog Items



Estimated Work



Team Commitment



Sprint Plan
Ready

1. INPUTS



Capstone Product Vision
High-level objectives and success criteria



Product Backlog
Prioritized list of features, stories, tasks, and NFRs



Team Capacity
Team availability and skills for the sprint



Constraints & Dependencies
Technical, resource, and external dependencies



Definition of Done
Quality standards and completion criteria

2. SPRINT PLANNING ACTIVITIES



1. Set the Sprint Goal

- Define what the team aims to achieve in this sprint
- Align with capstone objectives



2. Review & Select Backlog Items

- Review top-priority items
- Discuss scope and clarify requirements



3. Estimate the Work

- Break down into tasks
- Estimate using story points or hours



4. Plan the Work

- Decide how the work will be done
- Identify tasks, owners, and dependencies



5. Confirm Commitment

- Validate capacity vs. commitment
- Finalize sprint backlog

3. SPRINT PLAN OUTPUTS



SPRINT GOAL

A short, clear statement of what the team will achieve in the sprint.

SPRINT BACKLOG

ID	Backlog Item / Task	Estimate	Owner
US-101	Implement User Authentication	8	Alex
US-102	Build Product Catalog API	13	Priya
US-103	Develop React Product List	8	Sam
US-104	Cart & Checkout Flow	13	Jordan
US-105	Write Unit & Integration Tests	8	Taylor
...	...	...	...

PLAN SUMMARY



KEY DETAILS

Sprint: Sprint 1 Duration: 2 Weeks Team Size: 6 Total Points: 50 Capacity: 50

4. SPRINT PLANNING CHECKLIST



Sprint goal is clear and aligned with capstone vision



Backlog items are prioritized and well understood



Stories are broken down into actionable tasks



All items are estimated



Dependencies and risks are identified



Team has the skills and capacity to deliver



Commitment is confirmed



Sprint backlog is created and communicated

CAPSTONE FOCUS AREAS



Deliver End-to-End
Working Increment



Meet Quality
Standards



Collaborate &
Communicate



Manage Risks &
Dependencies



Continuously Integrate
& Deploy

TIPS FOR SUCCESS

- Keep the sprint goal specific and achievable.
- Focus on planned scope; avoid overload.
- Inspect and adapt in the Daily Scrum.
- Review and Retrospect to improve.

IDENTIFYING TECHNICAL / DELIVERY RISKS

Unscoreable risks surface as Week 6 thrash and an incomplete Lab 52 defense.

TECHNICAL RISKS

Kafka consumer lag during the live demo.

PostgreSQL migration failure against a shared schema.

JWT misconfiguration -- deny-by-default not enforced.

Contract drift between the UI and the API.

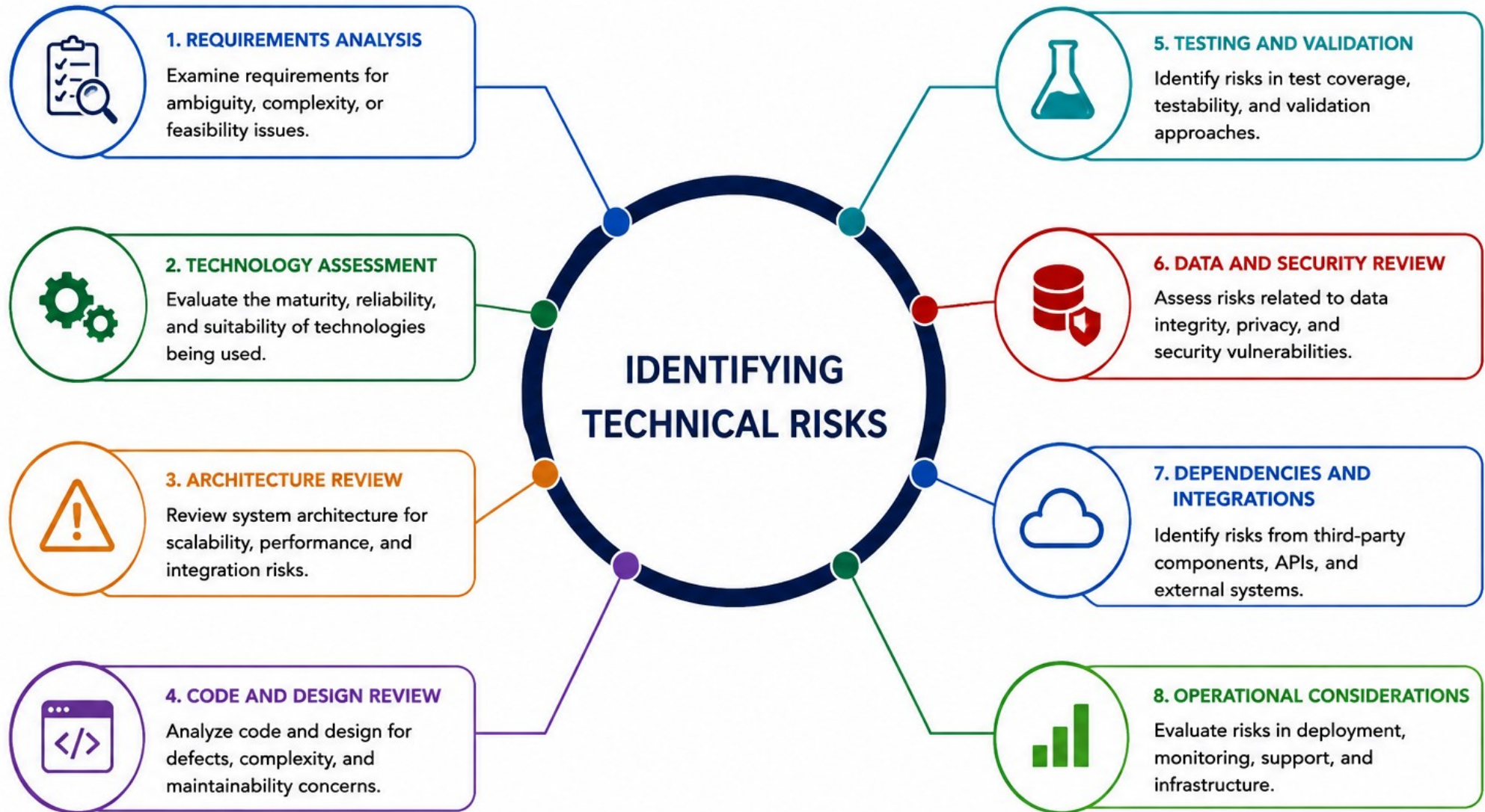
DELIVERY RISKS

- Pipeline secret leak in a committed file.
- Demo environment outage during Lab 52.
- A team member unavailable near a milestone.
- Scope creep past the frozen backlog.

Minimum risk set (Lab 48 requirement)

Kafka lag, PostgreSQL migration failure, JWT misconfig,
pipeline secret leak, demo outage, contract drift.

KEY TAKEAWAY 'Kafka might fail' with no score or owner cannot be prioritized -- every risk needs both before it counts.





RISK MITIGATION STRATEGIES

A tested control beats 'hope' every time -- mitigations must be as concrete as the risks they answer.

RISK REGISTER COLUMNS

id, risk, likelihood, impact, mitigation, owner, due date, status.

Score = likelihood x impact -- ranks risks, not feelings.

Every risk needs a trigger: what tells you it is happening.

Contingency: what you do if the mitigation itself fails.

WORKED MITIGATIONS

- Kafka lag -> bounded retries + DLT + a lag alert (Lab 49).
- Migration failure -> test the migration against a throwaway DB first.
- JWT misconfig -> negative tests for 401/403 in CI (Lab 51).
- Secret leak -> a pre-commit scan + a .gitignore review.

Risk row (excerpt)

```
Kafka lag | Likely | High | Bounded retry+DLT+alert | Lab 49 owner | Day 2
```

KEY TAKEAWAY 'Hope' is not a mitigation -- replace it with a testable control that has an owner and a due date.



TRY IT YOURSELF -- EXERCISE 5: OUTLINE RISK REGISTER

Reinforces: defining risk-register columns and three starter risks with owners and dates -- unowned risks fail professionalism.

STEPS (notes/lab48-risk-register.md)

Step	What To Do
1 -- Columns	id, risk, likelihood, impact, mitigation, owner, due date, status.
2 -- Starter Risks	Secret leak, Kafka lag during demo, incomplete JWT negative tests.
3 -- Check the Reference	Every risk needs owner + date -- unowned risks fail professionalism.
4 -- Scope	Outline only -- the full register is completed in Lab 48.

Starter risk row

Risk: _____

Likelihood: _____

Impact: _____

Mitigation: _____

Owner: _____

Due: _____

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 5 before continuing: exercises/exercise-05-risk-register.md (workspace: examples/module-48-exercises).

ARCHITECTURE DOCUMENTATION

ADRs and RFCs are how architecture decisions survive past the meeting where they were made.

Field	Requirement
Status	Proposed Accepted Superseded -- never left blank.
Owners	A named role or team -- never 'someone'.
Alternatives	At least two viable alternatives, or it is not a real decision.
Consequences	Benefits, costs, failure modes, and follow-up work.

REQUIRED ADR TOPICS



PostgreSQL as System of Record

ADR-001 -- justify the persistence choice.



Kafka for Interaction Events

ADR-002 -- justify async messaging over sync-only.



Consistency Strategy

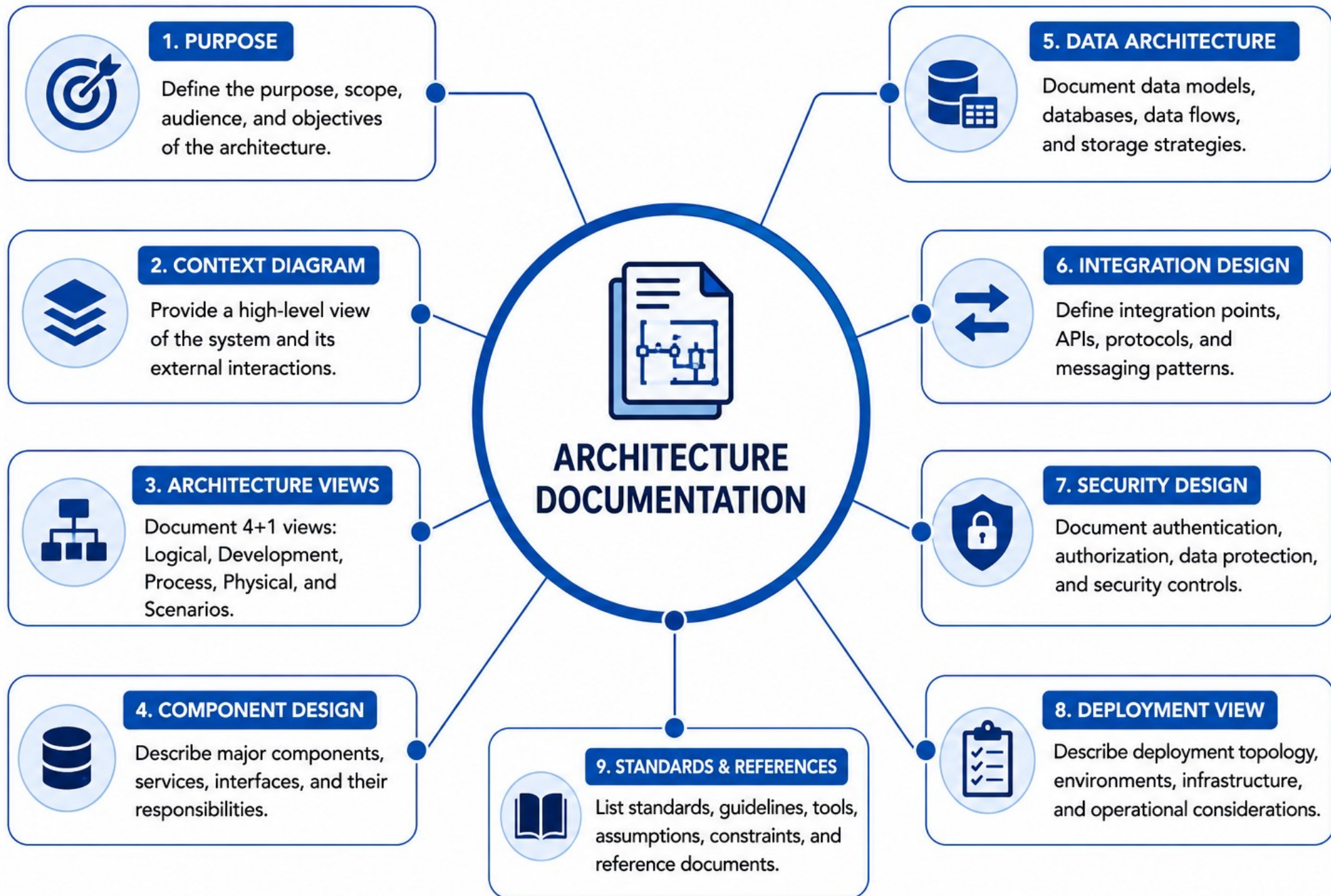
ADR-003 -- after-commit publish vs. outbox, with trade-offs.



JWT / OIDC Authentication

ADR-004 -- resource-server pattern, alternatives considered.

KEY TAKEAWAY Decision-only sticky notes are not ADRs -- expand alternatives and consequences, or the reviewer rejects the decision quality.



TRY IT YOURSELF -- EXERCISE 4: FILL ADR TOPIC TODOS

Reinforces: completing an ADR shortlist with blanks for status and owners across five required topics.

STEPS (notes/lab48-adr-todos.md)

Step	What To Do
1 -- Template	ADR title, status, decision needed by, options A/B, owner -- for each topic.
2 -- Fill Three	Fully fill three ADR stubs; leave two as title-only for Lab 48.
3 -- Consequence Reminder	Add: "Consequences must be written in Lab 48" under each stub.
4 -- No Code	Do not implement any of the decisions yet.

Five required ADR topics

```
API style, Kafka event versioning, authn/z approach,  
DB migration strategy, deploy target (k3s).
```

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 4 before continuing: `exercises/exercise-04-adr-todos.md` (workspace: `examples/module-48-exercises`).

PROJECT DOCUMENTATION STANDARDS

Six artifacts, one docs tree -- Labs 49-52 implement and defend against exactly these files.

THE SIX CORE ARTIFACTS

docs/architecture/context.md and container.md.

docs/nfrs.md -- measurable quality targets.

docs/backlog.md -- prioritized vertical stories.

docs/adrs/, docs/risk-register.md, and docs/team-plan.md.

DOCUMENTATION STANDARDS

- Prefer Mermaid/text diagrams in the repo over embedded images only.
- No secrets, real PII, kubeconfigs, or .env files, ever.
- A peer opens the docs alone and restates CAP-12 from them.
- Keep Accepted ADRs -- supersede, never silently delete.

docs/ tree (minimum)

```
docs/architecture/{context,container}.md, docs/nfrs.md,  
docs/backlog.md, docs/adrs/, docs/risk-register.md, docs/team-plan.md
```

KEY TAKEAWAY Embedded-image-only diagrams get ignored in review -- commit the Mermaid/text source alongside any picture.



TRY IT YOURSELF -- EXERCISE 6: PLANNING DOCS CHECKLIST

Capstone: build a Pass/Fail checklist mirroring the Week 6 planning portfolio -- the final gate before Lab 48 itself.

STEPS (notes/lab48-docs-checklist.md)

Step	What To Do
1 -- Rows	Context/container, measurable NFRs, backlog+AC, ADRs, risk register.
2 -- Paths	Map each row to a docs/... path from the Week 6 index.
3 -- Branch Note	Note the intended branch name pattern lab/48-crm.
4 -- Scope	Checklist warmup only -- do not claim Lab 48 complete.

Checklist row

Artifact: _____ Path: docs/_____ Peer OK? Y/N

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 6 before continuing: exercises/exercise-06-docs-checklist.md (workspace: examples/module-48-exercises). Final gate before Lab 48.

LAB OVERVIEW -- NORTHSTAR CRM EXECUTABLE PLAN

Turn the Northstar CRM brief into an executable architecture and delivery plan that Labs 49-52 implement against.

BUSINESS SCENARIO

- The capstone team must deliver one coherent Customer Management Platform, not five disconnected demonstrations. No Lab 49-52 work counts as in scope unless it maps to a backlog item, an ADR, and a measurable NFR or acceptance criterion.
- You own the planning gate for agent journeys around Amina (CUS-1001, ACTIVE) and Ravi (CUS-1002, PROSPECT to ACTIVE), interaction recording, search, profile, timeline, secure release, and final defense.
- A peer must be able to reproduce the intended Week 6 plan from your docs tree alone, without Slack archaeology.

LEARNING OBJECTIVES

- Produce C4 context and container views with trust boundaries.
- Write measurable NFRs with method, environment, and thresholds.
- Slice a prioritized vertical backlog including CAP-12.
- Record five or more ADRs with alternatives and consequences.
- Plan ownership, critical path, and a scored risk register.
- Keep CUS-1001/CUS-1002/lab-request-001 fixtures consistent throughout.

WHAT YOU BUILD

- customer-management-platform/docs/: architecture/context.md and container.md, nfrs.md, backlog.md, adrs/ (five or more), risk-register.md, and team-plan.md.

KEY TAKEAWAY Fixtures CUS-1001 (Amina Khan) and CUS-1002 (Ravi Singh) are synthetic -- never real customer PII. Duration: about 45 minutes with starters, up to 5-6 hours full path. Advanced capstone difficulty.

LAB TASKS AND DELIVERABLES

lab48-crm -- implementation steps, deliverables, and the evaluation rubric.

#	Implementation Step
1	Clarify product outcome: users, journeys, exclusions, success measures.
2	Model system context: C4 context with protocols and trust boundaries.
3	Design containers and data flow: React, Spring Boot, PostgreSQL, Kafka, IdP.
4	Define domain and contracts: endpoint and event sketches, versioning policy.
5	Write measurable NFRs: latency, availability, security, a11y, retention.
6	Create the prioritized vertical backlog, including CAP-12.
7	Record five or more ADRs: PostgreSQL, Kafka, consistency, JWT, deploy target.
8	Plan delivery and risk: team-plan.md and a scored risk-register.md.

EXPECTED DELIVERABLES

- docs/architecture/context.md and container.md
- docs/nfrs.md
- docs/backlog.md (at least one vertical story including CAP-12)
- docs/adrs/ (five or more ADRs)
- docs/risk-register.md and docs/team-plan.md

RUBRIC (100 MARKS)

- Environment/Structure 10 * Core Impl 30 * Integration/Config 15 * Failure Handling 15 * Verification 10 * Security/Production Awareness 10 * Docs 10

KEY TAKEAWAY Pretty diagrams without trust boundaries or owners lose architecture marks. A backlog that cannot demo the CUS-1001 interaction is incomplete core implementation.

MODULE 48 SUMMARY

In this module, we turned the Northstar CRM brief into an executable plan: architecture, NFRs, backlog, ADRs, and a scored risk register.

KEY CONCEPTS COVERED

**C4 Architecture**
Context and container views with trust boundaries -- no internals in context.

**Measurable NFRs**
Threshold + method + environment -- adjectives are banned.

**Vertical Backlog**
CAP-12 and friends -- ordered by value, risk, dependency, and learning.

**ADRs**
Status, owners, two or more alternatives, and consequences -- for every real decision.

**Scored Risk Register**
Likelihood x impact, trigger, mitigation, owner, due date.

**Team Plan**
Named owners for Labs 49-52 and a visible critical path.

MODULE FLOW RECAP



KEY TAKEAWAY Planning is a graded engineering skill: architecture, measurable NFRs, a vertical backlog, and owned risk -- reproducible by a peer without Slack archaeology.

KNOWLEDGE CHECK (1 OF 2)

Test your understanding of C4 views, NFRs, vertical backlogs, and ADRs.

1. What must a C4 context diagram avoid including?

- A. External systems and people
- B. Protocols and trust boundaries
- C. Class names and Kafka topic internals**
- D. The Identity Provider

3. What makes a backlog story 'vertical' rather than 'horizontal'?

- A. It only touches the database layer
- B. It crosses API, UI, data, and events lightly for one outcome**
- C. It is written by the Product Owner only
- D. It has no acceptance criteria

2. Why is 'the system should be fast' an unacceptable NFR?

- A. It is too short
- B. It has no threshold, measurement method, or environment**
- C. It does not mention Kafka
- D. It is grammatically incorrect

4. What does an ADR require besides a decision and a status?

- A. A single mandatory alternative
- B. Nothing else is needed
- C. Two or more alternatives and documented consequences**
- D. A cost estimate in dollars

KEY TAKEAWAY Context stays free of internals; NFRs need threshold + method + environment; backlogs stay vertical; ADRs need real alternatives.

KNOWLEDGE CHECK (2 OF 2)

Four more questions on risk registers, fixtures, traceability, and documentation standards.

5. What two things does every risk-register row require, besides likelihood and impact?

- A. A joke and a deadline
- B. A mitigation and a named owner**
- C. A screenshot only
- D. A Slack channel link

7. No Lab 49-52 work counts as 'in scope' unless it maps to what?

- A. A verbal agreement made in standup
- B. A backlog item, an ADR, and a measurable NFR or acceptance criterion**
- C. Whatever looks impressive in the demo
- D. The instructor's personal preference

6. Which fixture ID is reserved for not-found and negative-path testing?

- A. CUS-1001
- B. CUS-1002
- C. CUS-9999**
- D. lab-request-001

8. What must happen to a deferred backlog story instead of silently dropping it?

- A. Delete it from the repository
- B. Mark it Explicitly Deferred with an owner and a date**
- C. Leave it undocumented
- D. Move the discussion to a private chat

KEY TAKEAWAY Score every risk with likelihood, impact, mitigation, and owner; keep fixtures consistent; trace every story to backlog + ADR + NFR; defer explicitly, never silently.

*"Freeze the plan before you freeze the code
-- architecture, ADRs, and a backlog
everyone can defend."*

MODULE 48 COMPLETE

Capstone Planning and Architecture

You can now produce C4 context and container views, write measurable NFRs, slice a prioritized vertical backlog including CAP-12, record ADRs with real alternatives, and plan a scored risk register -- the frozen plan Labs 49-52 build against.